

SOLIDWORKS API (.NET VBA C#) — Examples (11/50): Create_Edge_Flange_Example_CSharp Create_Forming_Tool_Feature_Example_CSha

SOLIDWORKS API Documentation (converted from CHM)

Create Edge Flange Example (C#)

This example shows how to create a sheet metal edge flange.

```
// -----
// Preconditions: Open install_dir\samples\tutorial\sheetmetal\formtools\cover.sldprt.
//
// Postconditions: Creates Edge-Flange1 in the FeatureManager design tree.
// -----
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace CreateEdgeFlange_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {

            ModelDoc2 swModel;
            bool bValue;
            Edge swEdge;
            double dAngle;
            double dLength;
            Feature swFeature;
            Entity swEntity;
            Sketch swSketch;
            object[] vSketchSegments = new object[1];
            SketchLine swSketchLine;
            SketchPoint swStartPoint;
            SketchPoint swEndPoint;
            double dSize;
            double dFactor1;
            double dFactor2;
            Edge[] aFlangeEdges = new Edge[1];
            object[] vFlangeEdges = new object[1];
            Sketch[] aSketchFeats = new Sketch[1];
            object[] vSketchFeats = new object[1];
            EdgeFlangeFeatureData FeatData;
            Feature edgeFlangeFeat;
            CustomBendAllowance PBendData;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Flange angle
            dAngle = (90.0 / 180.0) * 3.1415926535897;

            // Flange length
            dLength = 0.02;

            swModel.ShowNamedView2("**Back", -1);
```

```

swModel.ViewZoomtofit2();

// Select edge
bValue = swModel.Extension.SelectByRay(0.0353852695288734, 0.0527495553160953, 0.04852679999999905, 0, 0, 1,
0.000283299018635423, 1, false, 0, 0);
swEdge = (Edge)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);

// Insert a sketch of the edge flange
swFeature = (Feature)swModel.InsertSketchForEdgeFlange(swEdge, dAngle, false);
bValue = swFeature.Select2(false, 0);

swModel.EditSketch();
swSketch = (Sketch)swModel.GetActiveSketch2();

swEntity = (Entity)swEdge;

// Select edge
bValue = swEntity.Select4(false, null);

// Use the edge in the sketch
bValue = swModel.SketchManager.SketchUseEdge(false);

// Get the created sketch line
vSketchSegments = (Object[])swSketch.GetSketchSegments();
swSketchLine = (SketchLine)vSketchSegments[0];

// Get start and end point
swStartPoint = (SketchPoint)swSketchLine.GetStartPoint2();
swEndPoint = (SketchPoint)swSketchLine.GetEndPoint2();

// Create additional lines to define sketch

// Set parameters defining the sketch geometry
dSize = swEndPoint.X - swStartPoint.X;
dFactor1 = 0.1;
dFactor2 = 1.25;

// Add directly and do not display to speed things up
swModel.SetAddToDB(true);
swModel.SetDisplayWhenAdded(false);

swModel.CreateLine2(swStartPoint.X, swStartPoint.Y, 0.0, swStartPoint.X, swStartPoint.Y + dLength, 0.0);
swModel.CreateLine2(swStartPoint.X, swStartPoint.Y + dLength, 0.0, swStartPoint.X + dFactor1 * dSize,
swStartPoint.Y + dFactor2 * dLength, 0.0);
swModel.CreateLine2(swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0, swEndPoint.X -
dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0);
swModel.CreateLine2(swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0, swEndPoint.X,
swEndPoint.Y + dLength, 0.0);
swModel.CreateLine2(swEndPoint.X, swEndPoint.Y, 0.0, swEndPoint.X, swEndPoint.Y + dLength, 0.0);

swModel.SetDisplayWhenAdded(true);
swModel.SetAddToDB(false);

// Commit changes made to the sketch
swModel.InsertSketch2(true);

// Insert the edge flange

aFlangeEdges[0] = swEdge;
aSketchFeats[0] = swSketch;

vFlangeEdges = aFlangeEdges;
vSketchFeats = aSketchFeats;

```

```

FeatData = (EdgeFlangeFeatureData)swModel.FeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmEdgeFlange);

FeatData.AddEdges(vFlangeEdges, vSketchFeats);

FeatData.UseDefaultBendRadius = false;
FeatData.BendRadius = 0.0007366;
FeatData.GapDistance = 0.001;
FeatData.BendAngle = dAngle;
FeatData.LockAngle = true;
FeatData.OffsetType = (int)swFlangeOffsetTypes_e.swFlangeOffsetBlind;
FeatData.OffsetDistance = dLength;
FeatData.OffsetDimType = (int)swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp;
FeatData.PositionType = (int)swFlangePositionTypes_e.swFlangePositionTypeMaterialInside;
FeatData.UsePositionOffset = true;
FeatData.PositionOffsetType = (int)swFlangeOffsetTypes_e.swFlangeOffsetBlind;
FeatData.PositionOffsetDistance = 0.01;
FeatData.UseDefaultBendAllowance = false;
PBendData = FeatData.GetCustomBendAllowance();
PBendData.KFactor = 0.5;
PBendData.Type = (int)swBendAllowanceTypes_e.swBendAllowanceDeduction;
FeatData.SetCustomBendAllowance(PBendData);
FeatData.UseDefaultBendRelief = false;
FeatData.UseReliefRatio = true;
FeatData.ReliefRatio = 0.5;
FeatData.AutoReliefType = (int)swSheetMetalReliefTypes_e.swSheetMetalReliefTear;
FeatData.ReliefTearType = (int)swReliefTearTypes_e.swReliefTearTypeRip;

edgeFlangeFeat = swModel.FeatureManager.CreateFeature(FeatData);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create Edge Flange Example (VBA)

This example shows how to create an edge flange.

```
'-----
'
' Preconditions: Sheet metal part is open and
'               contains an edge flange. Flat pattern is suppressed.
'
' Postconditions: Another edge flange is created.
'
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swEntity As SldWorks.Entity
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeat As SldWorks.Feature
Dim swEdgeFlange As SldWorks.EdgeFlangeFeatureData
Dim pNewEdge(0) As Object
Dim boolstatus As Boolean
Dim bRet As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    ' Select the edge for the new flange
    boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.06071758265887, -0.03627116313436, 0.05999086611155, False, 0, Nothing, 0)
    Set pNewEdge(0) = swSelMgr.GetSelectedObject6(1, 0)

    ' Select the sketch for the new flange
    boolstatus = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Set swEntity = swSelMgr.GetSelectedObject6(1, 0)

    ' Select the existing edge flange to get the edge flange feature
    boolstatus = swModelDocExt.SelectByID2("Edge-Flange1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True
    Set swEdgeFlange = swFeat.GetDefinition

    ' Roll back the model to modify it
    swEdgeFlange.AccessSelections swModel, Nothing

    swEntity.Select4 True, Nothing

    Dim vEdges As Variant
    vEdges = pNewEdge
    ' Create the new flange
    swEdgeFlange.Edges = vEdges

    ' Roll forward the model with the new flange
    swFeat.ModifyDefinition swEdgeFlange, swModel, Nothing

End Sub
```

Create Edge Flange Example (VB.NET)

This example shows how to create a sheet metal edge flange.

```
' -----
' Preconditions: Open install_dir\samples\tutorial\sheetmetal\formtools\cover.sldprt.
'
' Postconditions: Creates Edge-Flange1 in the FeatureManager design tree.
' -----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim bValue As Boolean
        Dim swEdge As Edge
        Dim dAngle As Double
        Dim dLength As Double
        Dim swFeature As Feature
        Dim swEntity As Entity
        Dim swSketch As Sketch
        Dim vSketchSegments(0) As Object
        Dim swSketchLine As SketchLine
        Dim swStartPoint As SketchPoint
        Dim swEndPoint As SketchPoint
        Dim dSize As Double
        Dim dFactor1 As Double
        Dim dFactor2 As Double
        Dim aFlangeEdges(0) As Edge
        Dim vFlangeEdges(0) As Object
        Dim aSketchFeats(0) As Sketch
        Dim vSketchFeats(0) As Object
        Dim FeatData As EdgeFlangeFeatureData
        Dim edgeFlangeFeat As Feature
        Dim PBendData As CustomBendAllowance
```

```
        swModel = swApp.ActiveDoc
```

```
        ' Flange angle
```

```
        dAngle = (90.0# / 180.0#) * 3.1415926535897
```

```
        ' Flange length
```

```
        dLength = 0.02
```

```
        swModel.ShowNamedView2("**Back", -1)
```

```
        swModel.ViewZoomtofit2
```

```
        ' Select edge for which to create the edge flange
```

```
        bValue = swModel.Extension.SelectByRay(0.0353852695288734, 0.0527495553160953, 0.0485267999999905, 0, 0, 1,
0.000283299018635423, 1, False, 0, 0)
```

```
        swEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)
```

```
        ' Insert a sketch of the edge flange profile
```

```
        swFeature = swModel.InsertSketchForEdgeFlange(swEdge, dAngle, False)
```

```
        bValue = swFeature.Select2(False, 0)
```

```

swModel.EditSketch
swSketch = swModel.GetActiveSketch2

swEntity = swEdge

' Select edge
bValue = swEntity.Select4(False, Nothing)

' Use the edge in the sketch
bValue = swModel.SketchManager.SketchUseEdge(False)

' Get the created sketch line
vSketchSegments = swSketch.GetSketchSegments
swSketchLine = vSketchSegments(0)

' Get start and end point
swStartPoint = swSketchLine.GetStartPoint2
swEndPoint = swSketchLine.GetEndPoint2

' Create additional lines to define sketch

' Set parameters defining the sketch geometry
dSize = swEndPoint.X - swStartPoint.X
dFactor1 = 0.1
dFactor2 = 1.25

' Add directly and do not display to speed things up
swModel.SetAddToDB(True)
swModel.SetDisplayWhenAdded(False)

swModel.CreateLine2(swStartPoint.X, swStartPoint.Y, 0#, swStartPoint.X, swStartPoint.Y + dLength, 0#)
swModel.CreateLine2(swStartPoint.X, swStartPoint.Y + dLength, 0#, swStartPoint.X + dFactor1 * dSize, swStartPoint.Y +
dFactor2 * dLength, 0#)
swModel.CreateLine2(swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#, swEndPoint.X - dFactor1 *
dSize, swStartPoint.Y + dFactor2 * dLength, 0#)
swModel.CreateLine2(swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#, swEndPoint.X, swEndPoint.Y +
dLength, 0#)
swModel.CreateLine2(swEndPoint.X, swEndPoint.Y, 0#, swEndPoint.X, swEndPoint.Y + dLength, 0#)

swModel.SetDisplayWhenAdded(True)
swModel.SetAddToDB(False)

' Commit changes made to the sketch
swModel.InsertSketch2(True)

' Insert the edge flange

aFlangeEdges(0) = swEdge
aSketchFeats(0) = swSketch

vFlangeEdges = aFlangeEdges
vSketchFeats = aSketchFeats

FeatData = swModel.FeatureManager.CreateDefinition(swFeatureNameID_e.swFmEdgeFlange)

Call FeatData.AddEdges(vFlangeEdges, vSketchFeats)

FeatData.UseDefaultBendRadius = False
FeatData.BendRadius = 0.0007366
FeatData.GapDistance = 0.001
FeatData.BendAngle = dAngle
FeatData.LockAngle = True
FeatData.OffsetType = swFlangeOffsetTypes_e.swFlangeOffsetBlind

```

```

FeatData.OffsetDistance = dLength
FeatData.OffsetDimType = swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp
FeatData.PositionType = swFlangePositionTypes_e.swFlangePositionTypeMaterialInside
FeatData.UsePositionOffset = True
FeatData.PositionOffsetType = swFlangeOffsetTypes_e.swFlangeOffsetBlind
FeatData.PositionOffsetDistance = 0.01
FeatData.UseDefaultBendAllowance = False
PBendData = FeatData.GetCustomBendAllowance
PBendData.KFactor = 0.5
PBendData.Type = swBendAllowanceTypes_e.swBendAllowanceDeduction
FeatData.SetCustomBendAllowance(PBendData)
FeatData.UseDefaultBendRelief = False
FeatData.UseReliefRatio = True
FeatData.ReliefRatio = 0.5
FeatData.AutoReliefType = swSheetMetalReliefTypes_e.swSheetMetalReliefTear
FeatData.ReliefTearType = swReliefTearTypes_e.swReliefTearTypeRip

```

```

edgeFlangeFeat = swModel.FeatureManager.CreateFeature(FeatData)

```

```

End Sub

```

```

''' <summary>

```

```

''' The SldWorks swApp variable is pre-assigned for you.

```

```

''' </summary>

```

```

Public swApp As SldWorks

```

```

End Class

```

Create Ellipse Example (C#)

This example shows how to create an ellipse circumscribing two sketch points.

```
//-----
// Preconditions: Open a part document that contains
// an active sketch that has two sketch points.
//
// Postconditions:
// 1. Creates an ellipse circumscribing the two sketch points.
// 2. Inspect Immediate window for ellipse point data, theta, and equation.
//-----
using Microsoft.VisualBasic;
using System;
using System.Windows.Forms;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateEllipse_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            MathUtility swMath = default(MathUtility);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swMath = (MathUtility)swApp.GetMathUtility();

            // Check whether document is active
            if (swModel == null)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            // Check whether document is a part
            int modelType = 0;
            modelType = swModel.GetType();

            if (modelType != (int)swDocumentTypes_e.swDocPART)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            // Select a plane on which to sketch
            if (SelectPlane(swModel) == false)
            {
                MessageBox.Show("Could not select plane.");
                return;
            }

            SketchManager swSkMgr = default(SketchManager);
            swSkMgr = swModel.SketchManager;

            Sketch swSketch = default(Sketch);
            swSketch = swSkMgr.ActiveSketch;

            // Check whether a sketch is active
            if (swSketch == null)
            {
                the macro again.");
                return;
            }

            const double pi = 3.1415926;

            // Get data from the points
            SketchPoint CtrPt = null;
            SketchPoint MajPt = null;

            swSkMgr = swModel.SketchManager;
            swSketch = swSkMgr.ActiveSketch;

            object[] vPoint = null;

            int i = 0;

            // Make sure the sketch is active
```

```

if (swSketch == null)
{
    MessageBox.Show("Please place the sketch in edit mode and run the macro again.");
    return;
}

// Make sure that at least two sketch points exist to define the position of the ellipse and its major axis
vPoint = (Object[])swSketch.GetSketchPoints2();
if (vPoint.GetUpperBound(0) == 0)
{
    MessageBox.Show("Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again.");
    return;
}

for (i = 0; i <= vPoint.GetUpperBound(0); i++)
{
    if (i == 0)
    {
        CtrPt = (SketchPoint)vPoint[i];
    }

    if (i == 1)
    {
        MajPt = (SketchPoint)vPoint[i];
    }
}

MathVector MajVec = default(MathVector);
double[] dirArr = new double[3];

dirArr[0] = MajPt.X - CtrPt.X;
dirArr[1] = MajPt.Y - CtrPt.Y;
dirArr[2] = 0;

MajVec = (MathVector)swMath.CreateVector((dirArr));
MathVector MajVecunit = default(MathVector);

MajVecunit = (MathVector)MajVec.Normalise();
MathVector normVec = default(MathVector);

dirArr[0] = 0;
dirArr[1] = 0;
dirArr[2] = 1;

normVec = (MathVector)swMath.CreateVector((dirArr));

MathVector MinVecunit = default(MathVector);
MathVector MinVec = default(MathVector);
double u = 0.0;

MinVecunit = (MathVector)normVec.Cross(MajVecunit);

double minlength = 50.0;
u = minlength / 1000.0;

MinVec = (MathVector)MinVecunit.Scale(u);

// Ensure that the minor axis is less than the major axis so that
// the equation returned is correct
if (MajVec.GetLength() < MinVec.GetLength())
{
    MessageBox.Show("The length of the minor axis must be less than that of the major axis.");
    return;
}

double[] pointvar;
Object pointobj;
pointobj = MinVec.ArrayData;
pointvar = (Double[])pointobj;

if (!(CtrPt.Z == 0) | !(MajPt.Z == 0) | !(pointvar[2] == 0))
{
    MessageBox.Show("Only 2D sketch is allowed.");
    return;
}

// Sketch the ellipse
SketchEllipse SkEllipse = default(SketchEllipse);
SkEllipse = (SketchEllipse)swModel.SketchManager.CreateEllipse(CtrPt.X, CtrPt.Y, 0, MajPt.X, MajPt.Y, 0, CtrPt.X +
pointvar[0], CtrPt.Y + pointvar[1], 0);

swModel.ViewZoomtofit2();

// Check that the sketch is an ellipse
Object vEllipse = null;
vEllipse = swSketch.GetEllipses3();

if (swSketch.GetEllipseCount() == 0)
{
    MessageBox.Show("An ellipse was not created. Please make sure that the sketch contains two points.");
    return;
}

// Extract information about the ellipse
SketchPoint swStartPt = default(SketchPoint);

```

```

SketchPoint swEndPt = default(SketchPoint);
SketchPoint swCtrPt = default(SketchPoint);
SketchPoint swMajPt = default(SketchPoint);
SketchPoint swMinPt = default(SketchPoint);

swStartPt = (SketchPoint)SkEllipse.GetStartPoint2();
swEndPt = (SketchPoint)SkEllipse.GetEndPoint2();
swCtrPt = (SketchPoint)SkEllipse.GetCenterPoint2();
swMajPt = (SketchPoint)SkEllipse.GetMajorPoint2();
swMinPt = (SketchPoint)SkEllipse.GetMinorPoint2();

Debug.Print("Sketch points");
Debug.Print("      Start Point   = (" + swStartPt.X * 1000.0 + ", " + swStartPt.Y * 1000.0 + ", " + swStartPt.Z *
1000.0 + ") mm");
Debug.Print("      End Point   = (" + swEndPt.X * 1000.0 + ", " + swEndPt.Y * 1000.0 + ", " + swEndPt.Z * 1000.0 + ")
mm");
Debug.Print("      Center Point = (" + swCtrPt.X * 1000.0 + ", " + swCtrPt.Y * 1000.0 + ", " + swCtrPt.Z * 1000.0 + ")
mm");
Debug.Print("      Major Point  = (" + swMajPt.X * 1000.0 + ", " + swMajPt.Y * 1000.0 + ", " + swMajPt.Z * 1000.0 + ")
mm");
Debug.Print("      Minor Point  = (" + swMinPt.X * 1000.0 + ", " + swMinPt.Y * 1000.0 + ", " + swMinPt.Z * 1000.0 + ")
mm");

// Algebraic equation for the ellipse
double h = 0;
double k = 0;
double a = 0;
double b = 0;
double theta = 0;

h = swCtrPt.X;
k = swCtrPt.Y;
a = 1 / (Math.Pow((swMajPt.X - swCtrPt.X), 2) + Math.Pow((swMajPt.Y - swCtrPt.Y), 2));
b = 1 / (Math.Pow((swMinPt.X - swCtrPt.X), 2) + Math.Pow((swMinPt.Y - swCtrPt.Y), 2));

// Return the tipping angle, theta; avoid divide by zero
if (swMajPt.X != swCtrPt.X)
{
    theta = Math.Atan((swMajPt.Y - swCtrPt.Y) / (swMajPt.X - swCtrPt.X));
}
else
{
    theta = pi / 2;
}

Debug.Print("Theta of ellipse: " + theta);

double c1 = 0;
double c2 = 0;
double c3 = 0;
double c4 = 0;
double c5 = 0;
double c6 = 0;

c1 = Math.Round((a * Math.Pow((Math.Cos(theta)), 2)) + (b * Math.Pow((Math.Sin(theta)), 2)), 2);
c2 = Math.Round((a - b) * Math.Sin(2 * theta), 2);
c3 = Math.Round((b * Math.Pow((Math.Cos(theta)), 2)) + (a * Math.Pow((Math.Sin(theta)), 2)), 2);
c4 = Math.Round((-2 * a * h * Math.Cos(theta)) + (2 * b * k * Math.Sin(theta)), 2);
c5 = Math.Round((-2 * a * h * Math.Sin(theta)) - (2 * b * k * Math.Cos(theta)), 2);
c6 = Math.Round(1 - (b * (Math.Pow(k, 2))) - (a * (Math.Pow(h, 2))), 2);

Debug.Print("Equation of the ellipse: " + c1 + "x^2 + " + c2 + "xy + " + c3 + "y^2 + " + c4 + "x + " + c5 + "y = " +
c6);

// NOTE: The coefficients of x and y in this
// equation represent a translation in the x-y plane.
// If they are 0, then the ellipse was not translated.
// Similarly the coefficient of xy represents
// a rotation. If it is 0, then the ellipse
// was not rotated.
//
// The units on each axis are in meters.
}

public bool SelectPlane(ModelDoc2 Plane)
{
    bool functionReturnValue = false;

    Feature featureTemp = default(Feature);
    featureTemp = (Feature)Plane.FirstFeature();

    while ((featureTemp != null))
    {
        string sFeatureName = null;
        sFeatureName = featureTemp.GetTypeName2();
        if (sFeatureName == "RefPlane")
        {
            featureTemp.Select2(true, 0);
            functionReturnValue = true;
            return functionReturnValue;
        }

        featureTemp = (Feature)featureTemp.GetNextFeature();
    }

    return functionReturnValue;
}

```

```
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Create Ellipse Example (VBA)

This example shows how to create an ellipse circumscribing two sketch points.

```
'-----
' Preconditions: Open a part document that contains
' an active sketch that has two sketch points.
'
```

```
' Postconditions:
' 1. Creates an ellipse circumscribing the two sketch points.
' 2. Inspect Immediate window for ellipse point data, theta, and equation.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swMath As SldWorks.MathUtility

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swMath = swApp.GetMathUtility
```

```
' Check whether document is active
```

```
If swModel Is Nothing Then
    swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
    Exit Sub
End If
```

```
' Check whether document is a part
```

```
Dim modelType As Long
modelType = swModel.GetType
```

```
If modelType <> SwConst.swDocPART Then
    swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
    Exit Sub
End If
```

```
' Select a plane on which to sketch
```

```
If SelectPlane(swModel) = False Then
    MsgBox "Could not select plane."
    Exit Sub
End If
```

```
Dim swSkMgr As SldWorks.SketchManager
Set swSkMgr = swModel.SketchManager
```

```
Dim swSketch As Sketch
Set swSketch = swSkMgr.ActiveSketch
```

```
' Check whether a sketch is active
```

```
If swSketch Is Nothing Then
    MsgBox "Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again."
    Exit Sub
End If
```

```
Const pi As Double = 3.1415926
```

```
' Get data from the points
```

```
Dim CtrPt As SldWorks.SketchPoint
Dim MajPt As SldWorks.SketchPoint
```

```
Set swSkMgr = swModel.SketchManager
Set swSketch = swSkMgr.ActiveSketch
```

```
Dim vPoint As Variant
```

```
Dim i As Long
```

```
' Make sure the sketch is active
```

```
If swSketch Is Nothing Then
    MsgBox "Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again."
    Exit Sub
End If
```

```
' Make sure that at least two sketch points exist to define the position of the ellipse and its major axis
```

```
vPoint = swSketch.GetSketchPoints2
```

```
If UBound(vPoint) = 0 Then
```

```
    MsgBox "Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again."
    Exit Sub
End If
```

```
For i = 0 To UBound(vPoint)
```

```
    If i = 0 Then
        Set CtrPt = vPoint(i)
    End If
```

```
    If i = 1 Then
        Set MajPt = vPoint(i)
    End If
```

```
Next i
```

```

Dim MajVec As MathVector
Dim dirArr(2) As Double
dirArr(0) = MajPt.X - CtrPt.X
dirArr(1) = MajPt.Y - CtrPt.Y
dirArr(2) = 0
Set MajVec = swMath.CreateVector((dirArr))
Dim MajVecunit As MathVector
Set MajVecunit = MajVec.Normalise
Dim normVec As MathVector
dirArr(0) = 0
dirArr(1) = 0
dirArr(2) = 1
Set normVec = swMath.CreateVector((dirArr))
Dim MinVecunit As MathVector
Dim MinVec As MathVector
Dim u As Double
Set MinVecunit = normVec.Cross(MajVecunit)
Dim minlength As Long
minlength = 50
u = minlength / 1000
Set MinVec = MinVecunit.Scale(u)
' Ensure that the minor axis is less than the major axis so that
' the equation returned is correct
If MajVec.GetLength < MinVec.GetLength Then
    MsgBox "The length of the minor axis must be less than that of the major axis."
    Exit Sub
End If
If Not CtrPt.Z = 0 Or Not MajPt.Z = 0 Or Not MinVec.ArrayData(2) = 0 Then
    MsgBox "2D sketch only."
    Exit Sub
End If
' Sketch the ellipse
Dim SkEllipse As SketchEllipse
Set SkEllipse = swModel.SketchManager.CreateEllipse(CtrPt.X, CtrPt.Y, 0, MajPt.X, MajPt.Y, 0, CtrPt.X +
MinVec.ArrayData(0), CtrPt.Y + MinVec.ArrayData(1), 0)
swModel.ViewZoomtofit2
' Check that the sketch is an ellipse
Dim vEllipse As Variant
vEllipse = swSketch.GetEllipses3
If swSketch.GetEllipseCount = 0 Then
    MsgBox "An ellipse was not created. Please make sure that the sketch contains at least one ellipse."
    Exit Sub
End If
' Extract information about the ellipse
Dim swStartPt As SldWorks.SketchPoint
Dim swEndPt As SldWorks.SketchPoint
Dim swCtrPt As SldWorks.SketchPoint
Dim swMajPt As SldWorks.SketchPoint
Dim swMinPt As SldWorks.SketchPoint
Set swStartPt = SkEllipse.GetStartPoint2
Set swEndPt = SkEllipse.GetEndPoint2
Set swCtrPt = SkEllipse.GetCenterPoint2
Set swMajPt = SkEllipse.GetMajorPoint2
Set swMinPt = SkEllipse.GetMinorPoint2
Debug.Print "Sketch points"
Debug.Print "    Start Point    = (" & swStartPt.X * 1000# & ", " & swStartPt.Y * 1000# & ", " & swStartPt.Z * 1000# &
") mm"
Debug.Print "    End Point      = (" & swEndPt.X * 1000# & ", " & swEndPt.Y * 1000# & ", " & swEndPt.Z * 1000# & ") mm"
Debug.Print "    Center Point   = (" & swCtrPt.X * 1000# & ", " & swCtrPt.Y * 1000# & ", " & swCtrPt.Z * 1000# & ") mm"
Debug.Print "    Major Point    = (" & swMajPt.X * 1000# & ", " & swMajPt.Y * 1000# & ", " & swMajPt.Z * 1000# & ") mm"
Debug.Print "    Minor Point    = (" & swMinPt.X * 1000# & ", " & swMinPt.Y * 1000# & ", " & swMinPt.Z * 1000# & ") mm"
' Algebraic equation for the ellipse
Dim h As Double
Dim k As Double
Dim a As Double
Dim b As Double
Dim theta As Double
h = swCtrPt.X
k = swCtrPt.Y
a = 1 / ((swMajPt.X - swCtrPt.X) ^ 2 + (swMajPt.Y - swCtrPt.Y) ^ 2)
b = 1 / ((swMinPt.X - swCtrPt.X) ^ 2 + (swMinPt.Y - swCtrPt.Y) ^ 2)
' Never divide by 0 and return the tipping angle
If swMajPt.X <> swCtrPt.X Then
    theta = Atn((swMajPt.Y - swCtrPt.Y) / (swMajPt.X - swCtrPt.X))
Else
    theta = pi / 2
End If
Debug.Print "Theta of ellipse: " & theta
Dim c1 As Double
Dim c2 As Double
Dim c3 As Double
Dim c4 As Double

```

```

Dim c5 As Double
Dim c6 As Double

c1 = Round((a * (Cos(theta)) ^ 2) + (b * (Sin(theta)) ^ 2), 2)
c2 = Round((a - b) * Sin(2 * theta), 2)
c3 = Round((b * (Cos(theta)) ^ 2) + (a * (Sin(theta)) ^ 2), 2)
c4 = Round((-2 * a * h * Cos(theta)) + (2 * b * k * Sin(theta)), 2)
c5 = Round((-2 * a * h * Sin(theta)) - (2 * b * k * Cos(theta)), 2)
c6 = Round(1 - (b * (k ^ 2)) - (a * (h ^ 2)), 2)

Debug.Print "Equation of the ellipse: " & c1 & "x^2 + " & c2 & "xy + " & c3 & "y^2 + " & c4 & "x + " & c5 & "y = " & c6

' NOTE: The coefficients of x and y in this
' equation represent a translation in the x-y plane.
' If they are 0, then the ellipse was not translated.
' Similarly the coefficient of xy represents
' a rotation. If it is 0, then the ellipse
' was not rotated.
'
' The units on each axis are in meters.

End Sub

Public Function SelectPlane(Plane As SldWorks.ModelDoc2) As Boolean

Dim featureTemp As Feature
Set featureTemp = Plane.FirstFeature

While Not featureTemp Is Nothing
Dim sFeatureName As String
sFeatureName = featureTemp.GetTypeName2
If sFeatureName = "RefPlane" Then
featureTemp.Select2 True, 0
SelectPlane = True
Exit Function
End If

Set featureTemp = featureTemp.GetNextFeature
Wend

End Function

```

[Back to top](#)

Create Ellipse Example (VB.NET)

This example shows how to create an ellipse circumscribing two sketch points.

 ' Preconditions: Open a a part document that contains
 ' an active sketch that has two sketch points.
 '-----

' Postconditions:
 ' 1. Creates an ellipse circumscribing the two sketch points.
 ' 2. Inspect Immediate window for ellipse point data, theta, and equation.
 '-----

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports System.Math
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swMath As MathUtility
```

```
        swModel = swApp.ActiveDoc
        swMath = swApp.GetMathUtility
```

```
        ' Check whether document is active
```

```
        If swModel Is Nothing Then
            swApp.SendMsgToUser2("A part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)
            Exit Sub
        End If
```

```
        ' Check whether document is a part
```

```
        Dim modelType As Long
        modelType = swModel.GetType
```

```
        If modelType <> swDocumentTypes_e.swDocPART Then
            swApp.SendMsgToUser2("A part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)
            Exit Sub
        End If
```

```
        ' Select a plane on which to sketch
```

```
        If SelectPlane(swModel) = False Then
            MsgBox("Could not select plane.")
            Exit Sub
        End If
```

```
        Dim swSkMgr As SketchManager
        swSkMgr = swModel.SketchManager
```

```
        Dim swSketch As Sketch
        swSketch = swSkMgr.ActiveSketch
```

```
        ' Check whether a sketch is active
```

```
        If swSketch Is Nothing Then
            MsgBox("Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again.")
            Exit Sub
        End If
```

```
        Const pi As Double = 3.1415926
```

```
        ' Get data from the points
```

```
        Dim CtrPt As SketchPoint = Nothing
        Dim MajPt As SketchPoint = Nothing
```

```
        swSkMgr = swModel.SketchManager
        swSketch = swSkMgr.ActiveSketch
```

```
        Dim vPoint As Object
```

```
        Dim i As Integer
```

```
        ' Make sure the sketch is active
```

```
        If swSketch Is Nothing Then
            MsgBox("Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again.")
            Exit Sub
        End If
```

```
        ' Make sure that at least two sketch points exist to define the position of the ellipse and its major axis
```

```
        vPoint = swSketch.GetSketchPoints2
```

```
        If UBound(vPoint) = 0 Then
            MsgBox("Please sketch a point for the center point, sketch another point to define the major axis, and run the macro again.")
            Exit Sub
        End If
```

```

End If

For i = 0 To UBound(vPoint)
    If i = 0 Then
        CtrPt = vPoint(i)
    End If

    If i = 1 Then
        MajPt = vPoint(i)
    End If
Next i

Dim MajVec As MathVector
Dim dirArr(2) As Double

dirArr(0) = MajPt.X - CtrPt.X
dirArr(1) = MajPt.Y - CtrPt.Y
dirArr(2) = 0

MajVec = swMath.CreateVector((dirArr))
Dim MajVecunit As MathVector

MajVecunit = MajVec.Normalise
Dim normVec As MathVector

dirArr(0) = 0
dirArr(1) = 0
dirArr(2) = 1

normVec = swMath.CreateVector((dirArr))

Dim MinVecunit As MathVector
Dim MinVec As MathVector
Dim u As Double

MinVecunit = normVec.Cross(MajVecunit)

Dim minlength As Integer
minlength = 50
u = minlength / 1000

MinVec = MinVecunit.Scale(u)

' Ensure that the minor axis is less than the major axis so that
' the equation returned is correct
If MajVec.GetLength < MinVec.GetLength Then
    MsgBox("The length of the minor axis must be less than that of the major axis.")
    Exit Sub
End If

If Not CtrPt.Z = 0 Or Not MajPt.Z = 0 Or Not MinVec.ArrayData(2) = 0 Then
    MsgBox("2D sketch only.")
    Exit Sub
End If

' Sketch the ellipse
Dim SkEllipse As SketchEllipse
SkEllipse = swModel.SketchManager.CreateEllipse(CtrPt.X, CtrPt.Y, 0, MajPt.X, MajPt.Y, 0, CtrPt.X + MinVec.ArrayData(0),
CtrPt.Y + MinVec.ArrayData(1), 0)

swModel.ViewZoomtofit2()

' Check that the sketch is an ellipse
Dim vEllipse As Object
vEllipse = swSketch.GetEllipses3

If swSketch.GetEllipseCount = 0 Then
    MsgBox("An ellipse was not created. Please make sure that the sketch contains at least one ellipse.")
    Exit Sub
End If

' Extract information about the ellipse
Dim swStartPt As SketchPoint
Dim swEndPt As SketchPoint
Dim swCtrPt As SketchPoint
Dim swMajPt As SketchPoint
Dim swMinPt As SketchPoint

swStartPt = SkEllipse.GetStartPoint2
swEndPt = SkEllipse.GetEndPoint2
swCtrPt = SkEllipse.GetCenterPoint2
swMajPt = SkEllipse.GetMajorPoint2
swMinPt = SkEllipse.GetMinorPoint2

Debug.Print("Sketch points")
Debug.Print("    Start Point = (" & swStartPt.X * 1000.0# & ", " & swStartPt.Y * 1000.0# & ", " & swStartPt.Z * 1000.0#
& ") mm")
Debug.Print("    End Point = (" & swEndPt.X * 1000.0# & ", " & swEndPt.Y * 1000.0# & ", " & swEndPt.Z * 1000.0# & ")
mm")
Debug.Print("    Center Point = (" & swCtrPt.X * 1000.0# & ", " & swCtrPt.Y * 1000.0# & ", " & swCtrPt.Z * 1000.0# & ")
mm")
Debug.Print("    Major Point = (" & swMajPt.X * 1000.0# & ", " & swMajPt.Y * 1000.0# & ", " & swMajPt.Z * 1000.0# & ")
mm")
Debug.Print("    Minor Point = (" & swMinPt.X * 1000.0# & ", " & swMinPt.Y * 1000.0# & ", " & swMinPt.Z * 1000.0# & ")
mm")

```

```

' Algebraic equation for the ellipse
Dim h As Double
Dim k As Double
Dim a As Double
Dim b As Double
Dim theta As Double

h = swCtrPt.X
k = swCtrPt.Y
a = 1 / ((swMajPt.X - swCtrPt.X) ^ 2 + (swMajPt.Y - swCtrPt.Y) ^ 2)
b = 1 / ((swMinPt.X - swCtrPt.X) ^ 2 + (swMinPt.Y - swCtrPt.Y) ^ 2)

' Return the tipping angle, theta; avoid divide by zero
If swMajPt.X <> swCtrPt.X Then
    theta = Atan((swMajPt.Y - swCtrPt.Y) / (swMajPt.X - swCtrPt.X))
Else
    theta = pi / 2
End If

Debug.Print("Theta of ellipse: " & theta)

Dim c1 As Double
Dim c2 As Double
Dim c3 As Double
Dim c4 As Double
Dim c5 As Double
Dim c6 As Double

c1 = Round((a * (Cos(theta)) ^ 2) + (b * (Sin(theta)) ^ 2), 2)
c2 = Round((a - b) * Sin(2 * theta), 2)
c3 = Round((b * (Cos(theta)) ^ 2) + (a * (Sin(theta)) ^ 2), 2)
c4 = Round((-2 * a * h * Cos(theta)) + (2 * b * k * Sin(theta)), 2)
c5 = Round((-2 * a * h * Sin(theta)) - (2 * b * k * Cos(theta)), 2)
c6 = Round(1 - (b * (k ^ 2)) - (a * (h ^ 2)), 2)

Debug.Print("Equation of the ellipse: " & c1 & "x^2 + " & c2 & "xy + " & c3 & "y^2 + " & c4 & "x + " & c5 & "y = " & c6)

' NOTE: The coefficients of x and y in this
'       equation represent a translation in the x-y plane.
'       If they are 0, then the ellipse was not translated.
'       Similarly the coefficient of xy represents
'       a rotation. If it is 0, then the ellipse
'       was not rotated.
'
'       The units on each axis are in meters.

```

End Sub

Public Function SelectPlane(ByVal Plane As ModelDoc2) As Boolean

```

Dim featureTemp As Feature
featureTemp = Plane.FirstFeature

While Not featureTemp Is Nothing
    Dim sFeatureName As String
    sFeatureName = featureTemp.GetTypeName2
    If sFeatureName = "RefPlane" Then
        featureTemp.Select2(True, 0)
        SelectPlane = True
        Exit Function
    End If

    featureTemp = featureTemp.GetNextFeature
End While

```

End Function

Public swApp As SldWorks

End Class

Create Equation-driven Curve Example (C#)

This example shows how to create and modify an equation-driven curve.

```
//-----
// Preconditions:
// 1. Open a new part document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a 2D sketch and creates an equation-driven spline of
//    a sine curve.
// 2. Examine the graphics area, then Press F5.
// 3. Edits the curve and creates a cosine curve.
// 4. Reduces the number of points in the spline.
// 5. Examine the Immediate window and graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace SimplifySketchSpline_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swExt;
        SelectionMgr swSelMgr;
        bool boolstatus;
        string y;
        double r1;
        double r2;
        bool rad;
        double ang;
        double xOff;
        double yOff;
        bool LockStart;
        bool LockEnd;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            boolstatus = swExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.SketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            SketchSegment skSegment = default(SketchSegment);
            skSegment = (SketchSegment)swModel.SketchManager.CreateEquationSpline2("", "sin(x)", "", "0", "6.28", false, 0, 0, 0,
true,
            true);
            swModel.ViewZoomtofit2();

            System.Diagnostics.Debugger.Break();
            //Examine the graphics area, then press F5

            SketchSpline skSpline = default(SketchSpline);
            skSpline = (SketchSpline)skSegment;
            skSpline.GetEquationParameters(out y, out r1, out r2, out rad, out ang, out xOff, out yOff, out LockStart, out
LockEnd);

            Debug.Print("y: " + y);
            Debug.Print("range start: " + r1);
            Debug.Print("range end: " + r2);
            Debug.Print("radian?: " + rad);
            Debug.Print("ang offset: " + ang);
            Debug.Print("x offset: " + xOff);
            Debug.Print("y offset: " + yOff);
            Debug.Print("lock start: " + LockStart);
            Debug.Print("lock end: " + LockEnd);

            // Change spline to a cosine curve
            y = "cos(x)";
            skSpline.SetEquationParameters(y, r1, r2, rad, ang, xOff, yOff, LockStart, LockEnd);

            // Reduce the number of points in the spline
            skSpline.Simplify(0.0);

            swModel.SketchManager.InsertSketch(true);
        }
    }
}
```

```
    public SldWorks swApp;  
}  
}
```

Create Equation-driven Curve Example (VBA)

This example shows how to create and modify an equation-driven curve.

```
'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a 2D sketch and creates an equation-driven spline of
'    a sine curve.
' 2. Examine the graphics area, then Press F5.
' 3. Edits the curve and creates a cosine curve.
' 4. Reduces the number of points in the spline.
' 5. Examine the Immediate window and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim y As String
Dim r1 As Double
Dim r2 As Double
Dim rad As Boolean
Dim ang As Double
Dim xOff As Double, yOff As Double
Dim LockStart As Boolean, LockEnd As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    boolstatus = swExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.SketchManager.InsertSketch True
    swModel.ClearSelection2 True

    Dim skSegment As SketchSegment
    Set skSegment = swModel.SketchManager.CreateEquationSpline2("", "sin(x)", "", "0", "6.28", False, 0, 0, 0, True, True)
    swModel.ViewZoomtofit2

    Stop
    ' Examine the graphics area, then press F5

    Dim skSpline As SketchSpline
    Set skSpline = skSegment
    Call skSpline.GetEquationParameters(y, r1, r2, rad, ang, xOff, yOff, LockStart, LockEnd)

    Debug.Print "y: " & y
    Debug.Print "range start: " & r1
    Debug.Print "range end: " & r2
    Debug.Print "radian?: " & rad
    Debug.Print "ang offset: " & ang
    Debug.Print "x offset: " & xOff
    Debug.Print "y offset: " & yOff
    Debug.Print "lock start: " & LockStart
    Debug.Print "lock end: " & LockEnd

    ' Change spline to a cosine curve
    y = "cos(x)"
    Call skSpline.SetEquationParameters(y, r1, r2, rad, ang, xOff, yOff, LockStart, LockEnd)

    ' Reduce the number of points in the spline
    skSpline.Simplify 0#

    swModel.SketchManager.InsertSketch True

End Sub
```

Create Equation-driven Curve Example (VB.NET)

This example shows how to create and modify an equation-driven curve.

```

'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a 2D sketch and creates an equation-driven spline of
'    a sine curve.
' 2. Examine the graphics area, then Press F5.
' 3. Edits the curve and creates a cosine curve.
' 4. Reduces the number of points in the spline.
' 5. Examine the Immediate window and graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim boolstatus As Boolean
    Dim y As String
    Dim r1 As Double
    Dim r2 As Double
    Dim rad As Boolean
    Dim ang As Double
    Dim xOff As Double, yOff As Double
    Dim LockStart As Boolean, LockEnd As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        boolstatus = swExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.SketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        Dim skSegment As SketchSegment
        skSegment = swModel.SketchManager.CreateEquationSpline2("", "sin(x)", "", "0", "6.28", False, 0, 0, 0, True, True)
        swModel.ViewZoomtofit2()

        Stop
        ' Examine graphics area, then press F5

        Dim skSpline As SketchSpline
        skSpline = skSegment
        Call skSpline.GetEquationParameters(y, r1, r2, rad, ang, xOff, yOff, LockStart, LockEnd)

        Debug.Print("y: " & y)
        Debug.Print("range start: " & r1)
        Debug.Print("range end: " & r2)
        Debug.Print("radian?: " & rad)
        Debug.Print("ang offset: " & ang)
        Debug.Print("x offset: " & xOff)
        Debug.Print("y offset: " & yOff)
        Debug.Print("lock start: " & LockStart)
        Debug.Print("lock end: " & LockEnd)

        ' Change spline to a cosine curve
        y = "cos(x)"
        Call skSpline.SetEquationParameters(y, r1, r2, rad, ang, xOff, yOff, LockStart, LockEnd)

        ' Reduce the number of points in the spline
        skSpline.Simplify(0.0#)

        swModel.SketchManager.InsertSketch(True)

    End Sub

    Public swApp As SldWorks

End Class

```

Create Exploded Views of an Assembly Example (C#)

This example shows how to create multiple exploded views of an assembly.

```
//-----
// Preconditions: Open an assembly document.
//
// Postconditions:
// 1. Press F5 and inspect the Immediate window.
// 2. Press F5 and inspect the graphics area at all
//    subsequent System.Diagnostics.Debugger.Break() statements.
//-----

using Microsoft.VisualBasic;

using System;

using System.Collections;

using System.Collections.Generic;

using System.Data;

using System.Diagnostics;

using SolidWorks.Interop.sldworks;

using SolidWorks.Interop.swconst;

using System.Runtime.InteropServices;

namespace CreateExplodedViews_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        AssemblyDoc swAssembly;

        object[] vViewName;
        string sViewName;
        int i;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;

            // Create five exploded views
            for (i = 0; i <= 4; i++)
            {
                swAssembly.CreateExplodedView();
            }

            vViewName = (object[])swAssembly.GetExplodedViewNames();
            Debug.Print("Number of exploded views created: " + swAssembly.GetExplodedViewCount());

            for (i = 0; i <= vViewName.GetUpperBound(0); i++)
            {
                sViewName = (string)vViewName[i];

                Debug.Print("    Exploded
view name: " + sViewName);
            }

            System.Diagnostics.Debugger.Break();

            for (i = 0; i <= vViewName.GetUpperBound(0); i++)
            {
                sViewName = (string)vViewName[i];
```

```
        swAssembly.ShowExploded2(true, sViewName);
    }

    System.Diagnostics.Debugger.Break();
    for (i = 0; i <= vViewName.GetUpperBound(0); i++)
    {
        sViewName = (string)vViewName[i];
        swAssembly.ShowExploded2(false, sViewName);
    }

    System.Diagnostics.Debugger.Break();
    swAssembly.ViewExplodeAssembly();

    System.Diagnostics.Debugger.Break();
    swAssembly.ViewCollapseAssembly();
}

public SldWorks swApp;
}
}
```

Create Exploded Views of an Assembly Example (VBA)

This example shows how to create multiple exploded views of an assembly.

```
'-----  
' Preconditions: Open an assembly document.  
'  
' Postconditions:  
' 1. Press F5 and inspect the Immediate window.  
' 2. Press F5 and inspect the graphics area at all  
'    subsequent Stop statements.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim vViewName As Variant  
Dim sViewName As String  
Dim i As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swAssembly = swModel  
  
    ' Create five exploded views  
    For i = 0 To 4  
        swAssembly.CreateExplodedView  
    Next i  
  
    vViewName = swAssembly.GetExplodedViewNames  
    Debug.Print "Number of exploded views created: " & swAssembly.GetExplodedViewCount  
  
    For i = 0 To UBound(vViewName)  
        sViewName = vViewName(i)  
        Debug.Print " Exploded view name: " & sViewName  
    Next i  
  
    Stop  
    For i = 0 To UBound(vViewName)  
        sViewName = vViewName(i)  
        Call swAssembly.ShowExploded2(True, sViewName)  
    Next i  
  
    Stop  
    For i = 0 To UBound(vViewName)  
        sViewName = vViewName(i)  
        Call swAssembly.ShowExploded2(False, sViewName)  
    Next i  
  
    Stop  
    swAssembly.ViewExplodeAssembly  
  
    Stop  
    swAssembly.ViewCollapseAssembly  
End Sub
```

Create Exploded Views of an Assembly Example (VB.NET)

This example shows how to create multiple exploded views of an assembly.

```
'-----  
' Preconditions: Open an assembly document.  
'  
' Postconditions:  
' 1. Press F5 and inspect the Immediate window.  
' 2. Press F5 and inspect the graphics area at all  
'    subsequent Stop statements.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System.Runtime.InteropServices  
  
Imports System  
  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim swModel As ModelDoc2  
  
    Dim swAssembly As AssemblyDoc  
  
    Dim vViewName As Object  
  
    Dim sViewName As String  
  
    Dim i As Integer  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
  
        swAssembly = swModel  
  
        ' Create five exploded views  
  
        For i = 0 To 4  
  
            swAssembly.CreateExplodedView()  
  
        Next i  
  
        vViewName = swAssembly.GetExplodedViewNames  
  
        Debug.Print("Number of  
exploded views created: " & swAssembly.GetExplodedViewCount)  
  
        For i = 0 To UBound(vViewName)  
  
            sViewName = vViewName(i)  
  
            Debug.Print(" Exploded  
view name: " & sViewName)  
  
        Next i  
  
        Stop  
  
        For i = 0 To UBound(vViewName)  
  
            sViewName = vViewName(i)  
  
            Call swAssembly.ShowExploded2(True, sViewName)  
  
        Next i  
  
        Stop  
  
        For i = 0 To UBound(vViewName)  
  
            sViewName = vViewName(i)  
  
            Call swAssembly.ShowExploded2(False, sViewName)  
  
        Next i  
  
        Stop  
  
    End Sub  
End Class
```

```
swAssembly.ViewExplodeAssembly()
```

```
Stop
```

```
swAssembly.ViewCollapseAssembly()
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Extrude Feature Using Sketch Contours Example (C#)

This example shows how to create an extrude feature using sketch contours.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a sketch containing sketch contours.
// 3. Creates a boss extrude feature using the sketch of sketch
//    contours.
// 4. Selects the boss extrude feature and accesses
//    its data.
// 5. Gets the sketch contours.
// 6. Get whether each sketch contour is open or closed.
// 7. Examine the FeatureManager design tree, graphics area, and
//    the Immediate window.
//-----
using System;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace ExtrudeFeatureData2CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeature = default(Feature);
            ExtrudeFeatureData2 swExtrudeFeatureData = default(ExtrudeFeatureData2);
            bool status = false;
            object[] skcontours = null;
            SketchContour skcontour = null;
            int nbrContours = 0;
            int i = 0;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2016\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExtension = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;

            //Create sketch containing sketch contours
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.010564, -0.041843, 0.0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.043155, 0.0, 0.0, 0.048428, -0.01221, 0.0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.043155, 0.0, 0.0, -0.043214, -0.014954, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Create boss extrude feature
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.047096875714166, 0.00724922083273226, 0.018531938896921, true, 0, null
, 0);
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", 0.0473122625955432, -0.015948285832011, -0.0155264330079864, true, 0, nul
l, 0);
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.00880361157802517, -0.0246473508312897, 0.0199951653548178, true, 0, n
ull, 0);
            swModel.ClearSelection2(true);
            swSelectionMgr.EnableContourSelection = true;
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.047096875714166, 0.00724922083273226, 0.018531938896921, true, 4, null
, 0);
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", 0.0473122625955432, -0.015948285832011, -0.0155264330079864, true, 4, nul
l, 0);
            status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.00880361157802517, -0.0246473508312897, 0.0199951653548178, true, 4, n
ull, 0);
            swFeature = swFeatureMgr.FeatureExtrusion3(true, false, false, 0, 0, 0.01016, 0.00254, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
0, 0, false);
            swSelectionMgr.EnableContourSelection = false;

            //Select the boss extrude feature
            status = swModelDocExtension.SelectByID("Boss-
Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);

            //Access the boss extrude feature data
            swExtrudeFeatureData = (ExtrudeFeatureData2)swFeature.GetDefinition();
            swExtrudeFeatureData.AccessSelections(swModel, null);

            //Get the sketch contours in the feature
            skcontours = (object[])swExtrudeFeatureData.Contours;
            nbrContours = swExtrudeFeatureData.GetContoursCount();
            Debug.Print("Number of sketch contours in this extrude feature: " + nbrContours);

            //Get the sketch contours in the extrude feature
            skcontours = (object[])swExtrudeFeatureData.Contours;

            //Get each sketch contour and whether it is open or closed
            for (i = 0; i <= (nbrContours - 1); i++)
            {
                skcontour = (SketchContour)skcontours[i];
                Debug.Print(" Sketch contour " + i + " is closed? " + skcontour.IsClosed());
            }

            //Releases selection access
            swExtrudeFeatureData.ReleaseSelectionAccess();
        }
    }
}
```

```
}  
  /// <summary>  
  /// The SldWorks swApp variable is pre-assigned for you.  
  /// </summary>  
  public SldWorks swApp;  
}
```

Create Extrude Feature Using Sketch Contours Example (VBA)

This example shows how to create an extrude feature using sketch contours.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a sketch containing sketch contours.
' 3. Creates a boss extrude feature using the sketch of sketch
'   contours.
' 4. Selects the boss extrude feature and accesses
'   its data.
' 5. Gets the sketch contours.
' 6. Get whether each sketch contour is open or closed.
' 7. Examine the FeatureManager design tree, graphics area, and
'   the Immediate window.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swModelDocExtension As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swExtrudeFeatureData As SldWorks.ExtrudeFeatureData2
Dim status As Boolean
Dim skcontours As Variant
Dim skcontour As SldWorks.SketchContour
Dim nbrContours As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExtension = swModel.Extension
    Set swSketchMgr = swModel.SketchManager
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeatureMgr = swModel.FeatureManager

    'Create sketch containing sketch contours
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.010564, -0.041843, 0#)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchMgr.CreateCircle(0.043155, 0#, 0#, 0.048428, -0.01221, 0#)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchMgr.CreateCircle(-0.043155, 0#, 0#, -0.043214, -0.014954, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Create boss extrude feature from sketch of sketch contours
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCHCONTOUR", -0.047096875714166, 7.24922083273226E-03, 0.018531938896921, True, 0, Nothing, 0)
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCHCONTOUR", 4.73122625955432E-02, -0.015948285832011, -1.55264330079864E-02, True, 0,
    Nothing, 0)
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCHCONTOUR", -8.80361157802517E-03, -2.46473508312897E-02, 1.99951653548178E-02, True, 0,
    Nothing, 0)
    swModel.ClearSelection2 True
    swSelectionMgr.EnableContourSelection = True
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCHCONTOUR", -0.047096875714166, 7.24922083273226E-03, 0.018531938896921, True, 4, Nothing, 0)
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCHCONTOUR", 4.73122625955432E-02, -0.015948285832011, -1.55264330079864E-02, True, 4,
    Nothing, 0)
    status = swModelDocExtension.SelectByID2("Sketch1", "SKETCHCONTOUR", -8.80361157802517E-03, -2.46473508312897E-02, 1.99951653548178E-02, True, 4,
    Nothing, 0)
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.01016, 0.00254, False, False, False, False, 1.74532925199433E-02,
    1.74532925199433E-02, False, False, False, True, True, True, 0, 0, False)
    swSelectionMgr.EnableContourSelection = False

    'Select the boss extrude feature
    status = swModelDocExtension.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swExtrudeFeatureData = swFeature.GetDefinition

    'Access the boss extrude feature data
    swExtrudeFeatureData.AccessSelections swModel, Nothing

    'Get the number of sketch contours in the extrude feature
    nbrContours = swExtrudeFeatureData.GetContoursCount
    Debug.Print "Number of sketch contours in the extrude feature: " & nbrContours

    'Get the sketch contours in the extrude feature
    skcontours = swExtrudeFeatureData.contours

    'Get each sketch contour and whether it is open or closed
    For i = 0 To (nbrContours - 1)
        Set skcontour = skcontours(i)
        Debug.Print " Sketch contour " & i & " is closed? " & skcontour.IsClosed
    Next i

```

```
'Release selection access  
swExtrudeFeatureData.ReleaseSelectionAccess
```

```
End Sub
```

Create Extrude Feature Using Sketch Contours Example (VB.NET)

This example shows how to create an extrude feature using sketch contours.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
Postconditions:
1. Opens a new part document.
2. Creates a sketch containing sketch contours.
3. Creates a boss extrude feature using the sketch of sketch
  contours.
4. Selects the boss extrude feature and accesses
  its data.
5. Gets the sketch contours.
6. Get whether each sketch contour is open or closed.
7. Examine the FeatureManager design tree, graphics area, and
  the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swModelDocExtension As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeatureMgr As FeatureManager
        Dim swFeature As Feature
        Dim swExtrudeFeatureData As ExtrudeFeatureData2
        Dim status As Boolean
        Dim skContours() As Object
        Dim skContour As SketchContour = Nothing
        Dim nbrContours As Integer
        Dim i As Integer

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
        swModelDocExtension = swModel.Extension
        swSketchMgr = swModel.SketchManager
        swSelectionMgr = swModel.SelectionManager
        swFeatureMgr = swModel.FeatureManager

        'Create sketch containing sketch contours
        swSketchMgr.InsertSketch(True)
        swSketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.010564, -0.041843, 0.0#)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchMgr.CreateCircle(0.043155, 0.0#, 0.0#, 0.048428, -0.01221, 0.0#)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchMgr.CreateCircle(-0.043155, 0.0#, 0.0#, -0.043214, -0.014954, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Create boss extrude feature
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.047096875714166, 0.00724922083273226, 0.018531938896921, True, 0, Nothing,
0)
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", 0.0473122625955432, -0.015948285832011, -0.0155264330079864, True, 0, Nothing,
, 0)
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.00880361157802517, -0.0246473508312897, 0.0199951653548178, True, 0, Nothi
ng, 0)
        swModel.ClearSelection2(True)
        swSelectionMgr.EnableContourSelection = True
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.047096875714166, 0.00724922083273226, 0.018531938896921, True, 4, Nothing,
0)
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", 0.0473122625955432, -0.015948285832011, -0.0155264330079864, True, 4, Nothing,
, 0)
        status = swModelDocExtension.SelectByID("Sketch1", "SKETCHCONTOUR", -0.00880361157802517, -0.0246473508312897, 0.0199951653548178, True, 4, Nothi
ng, 0)
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.01016, 0.00254, False, False, False, False, 0.0174532925199433, 0.017453292
5199433, False, False, False, False, True, True, True, 0, 0, False)
        swSelectionMgr.EnableContourSelection = False

        'Select the boss extrude feature
        status = swModelDocExtension.SelectByID("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swExtrudeFeatureData = swFeature.GetDefinition

        'Access the boss extrude feature data
        swExtrudeFeatureData.AccessSelections(swModel, Nothing)

        'Get the number of sketch contours in the extrude feature
        nbrContours = swExtrudeFeatureData.GetContoursCount
        Debug.Print("Number of sketch contours in the extrude feature: " & nbrContours)

        'Get the sketch contours in the extrude feature
        skContours = swExtrudeFeatureData.Contours

        'Get each sketch contour and whether it is open or closed
        For i = 0 To (nbrContours - 1)
            skContour = skContours(i)
            Debug.Print(" Sketch contour " & i & " is closed? " & skContour.IsClosed)
        Next i

        'Release selection access
        swExtrudeFeatureData.ReleaseSelectionAccess()

    End Sub

''' <summary>

```

```
''' The Sldworks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Create Extruded Thin Feature Example (VBA)

This example shows how to create an extruded thin feature using a variety of parameter values.

```
'-----
' Preconditions: None.
'
' Postconditions: When the macro stops, press F5 five more times to create
' five different thin extrusion features.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myFeature As SldWorks.Feature
Dim swPart As SldWorks.PartDoc
Dim nStartCondition As Long
Dim nEndcondition1 As Long
Dim nEndcondition2 As Long
Dim lCase As Long
Dim dStartOffset As Double
Dim bFlipOffset As Boolean
Dim dThickness As Double
Dim swExtrusionData As SldWorks.ExtrudeFeatureData2
Dim lThinFeatureType As Long
Dim lCapEnds As Long
Dim bValue As Boolean

Dim boolstatus As Boolean

Option Explicit
Sub main()
    Set swApp = Application.SldWorks

    'New model document
    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2018\templates\Part.prtdot", 0, swSheetWidth, swSheetHeight)

    Set swPart = Part
    Set Part = swApp.ActiveDoc

    'Set some parameters used for all cases
    nEndcondition1 = swEndCondBlind
    nEndcondition2 = swEndCondBlind
    dThickness = 0.001
    lThinFeatureType = 2 ' Mid-plane
    lCapEnds = 0

    'Create sketch
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -4.75777316877496E-02, 1.30140669704215E-02, 0.016264555537633,
    False, 0, Nothing, 0)
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True

    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-4.72149072628614E-02, 2.51029484316118E-02, 0, 6.72972660081509E-02,
    -4.56125914055245E-02, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True

    Part.ClearSelection2 True

    For lCase = 1 To 6
        Part.ClearSelection2 True

        bValue = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

        Select Case lCase
            Case 1
                nStartCondition = swStartSketchPlane
                dStartOffset = 0#
                bFlipOffset = False

            Case 2
                nStartCondition = swStartOffset
                dStartOffset = 0#
                bFlipOffset = False
```

```

Case 3
    nStartCondition = swStartOffset
    dStartOffset = 0#
    bFlipOffset = True

Case 4
    nStartCondition = swStartOffset
    dStartOffset = 0.01
    bFlipOffset = False

Case 5
    nStartCondition = swStartOffset
    dStartOffset = 0.01
    bFlipOffset = True

Case 6
    nStartCondition = swStartVertex
    dStartOffset = 0#
    bFlipOffset = True
    dThickness = 0.01

bValue = Part.Extension.SelectByID2("", "VERTEX", -0.05999251028807, 0.05489880658436, 0, True, 0, Nothing, 0)

End Select

Set myFeature = Part.FeatureManager.FeatureExtrusionThin2(True, False, False, nEndcondition1, nEndcondition2, 0.00254,
0.00254, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, dThickness,
0.00254, 0.00254, lThinFeatureType, lCapEnds, False, 0.005, True, True, nStartCondition, dStartOffset, bFlipOffset)

Debug.Print myFeature.Name & " [" & myFeature.GetTypeName & "]"

Set swExtrusionData = myFeature.GetDefinition

bValue = swExtrusionData.AccessSelections(Part, Nothing)

If (swExtrusionData.IsThinFeature = False) Then
    Debug.Print "ERROR: Must be a thin feature."
End If

Select Case swExtrusionData.FromType

    Case SwConst.swExtrudeFrom_e.swExtrudeFrom_SketchPlane

        Debug.Print " from: sketchplane"

    Case SwConst.swExtrudeFrom_e.swExtrudeFrom_Offset

        Debug.Print " from: offset"

        Debug.Print " distance = " & swExtrusionData.FromOffsetDistance
        Debug.Print " reverse = " & swExtrusionData.FromOffsetReverse

    Case SwConst.swExtrudeFrom_e.swExtrudeFrom_SurfaceFacePlane

        Debug.Print " from: surface"

    Case SwConst.swExtrudeFrom_e.swExtrudeFrom_Vertex

        Debug.Print " from: vertex"

End Select

swExtrusionData.ReleaseSelectionAccess

' Examine the feature
Stop
' Now delete the feature
' The sketch remains, so that you can select it again
bValue = myFeature.Select2(True, 0)

bValue = Part.Extension.DeleteSelection2(swDelete_Children)

Next lCase
End Sub

```

Create Face Plane Intersection Structure System Member (C#)

This example shows how to create a structure system member along the intersection of a face and a plane.

```
// =====
// Preconditions:
// 1. Ensure the specified part template exists.
// 2. Open an Immediate window.
// 3. Press F5.
//
// Postconditions:
// 1. Creates Boss-Extrude1 and Plane1 in a new part.
// 2. Selects a parameter face on the extrusion.
// 3. Selects Plane1.
// 4. Creates a face plane intersection member along the intersection of the selected face and plane.
// 5. Creates a Structure System1.
// 6. Inspect the member on Plane1 in the graphics area.
// 7. Press F5 to edit the structure system.
// 8. Gets all of the structure system members in all of the profile group folders.
// 9. Changes the member extension lengths in directions 1 and 2.
// 10. Selects the Right Plane to intersect the parameter face.
// 11. Modifies the structure system.
// 12. Inspect the member move to the Right Plane in the graphics area.
// =====
```

```
using System.Diagnostics;
using System.Runtime.InteropServices;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace PrimaryMemberFacePlaneIntersection_CSharp
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private ModelDocExtension modDocExt;
        private StructureSystemMemberFeatureData structMemDef;
        private StructureSystemMemberProfile profDef;
        private PrimaryStructuralMemberFeatureData PrimDef;
        private PrimaryMemberFacePlaneIntersectionFeatureData facePlaneIntrSecDef;
        private object[] faces = new object[1];
        private bool stat;
        private object[] IntSecPlane = new object[1];
        private StructureSystemMemberFeatureData[] memData = new StructureSystemMemberFeatureData[1];
        private object[] memDataArray;
        private Feature structSys;
        private StructureSystemFolder structSysModDef;
        private object[] outProfiles;
        private StructureSystemMemberFeatureData MemberData;
        private object[] memberArray;
        private Feature profileGrpFeat;
        private ProfileGroupFolder profileGrp;
        private StructureSystemMemberFeatureData memToChange;
        private PrimaryMemberFacePlaneIntersectionFeatureData faceIntrMemberToChange;
        private object[] newIntSecPlane = new object[1];
        private Feature feat;
```

```

private int I;
private int J;

public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize;
    ArraySize = Objects.GetUpperBound(0);
    var d = new Dispatcher[ArraySize + 1];
    int ArrayIndex;
    var loopTo = ArraySize;
    for (ArrayIndex = 0; ArrayIndex <= loopTo; ArrayIndex++)
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    return d;
}

public void Main()
{
    double swSheetWidth;
    swSheetWidth = 0d;
    double swSheetHeight;
    swSheetHeight = 0d;
    modDoc = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0,
swSheetWidth, swSheetHeight);
    PartDoc swPart;
    swPart = (PartDoc)modDoc;
    modDoc = (ModelDoc2)swApp.ActiveDoc;
    modDocExt = modDoc.Extension;
    swSelMgr = (SelectionMgr)modDoc.SelectionManager;
    stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.SketchManager.InsertSketch(true);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
(int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
    stat = modDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
(int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
    object[] vSkLines;
    vSkLines = (object[])modDoc.SketchManager.CreateCornerRectangle(-0.0168788132580097d, 0.0116544186781496d, 0,
0.0139317188796271d, -0.00736773594595662d, 0);
    modDoc.ShowNamedView2("*Trimetric", 8);
    modDoc.ViewZoomtofit2();
    Feature myFeature;
    myFeature = modDoc.FeatureManager.FeatureExtrusion2(false, false, false, 0, 0, 0.2d, 0.2d, false, false, false, false,
0.0174532925199433d, 0.0174532925199433d, false, false, false, false, true, true, true, 0, 0, false);
    swSelMgr.EnableContourSelection = false;
    stat = modDocExt.SelectByRay(0.199999999999932d, 0.00280649656951937d, 0.000880259028576802d, -0.400036026779314d,
-0.515038074910018d, -0.758094294050287d, 0.00111139990390966d, 2, false, 0, 0);
    RefPlane myRefPlane;
    myRefPlane = (RefPlane)modDoc.FeatureManager.InsertRefPlane(4, 0, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);

    // Configure a structure system face plane intersection member
    structMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(3);
    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
structMemDef.StructureSystemMemberType);
    structMemDef.StartEndExtendD1 = 0.4d;
    Debug.Print("Extend length in direction 1: " + structMemDef.StartEndExtendD1);
    structMemDef.StartEndExtendD2 = 0.15d;
    Debug.Print("Extend length in direction 2: " + structMemDef.StartEndExtendD2);
    profDef = (StructureSystemMemberProfile)structMemDef.MemberProfile;
    profDef.ProfileStandard = "ansi inch";
    profDef.ProfileType = "c channel";
    profDef.ProfileSize = "3 x 5";
    PrimDef = (PrimaryStructuralMemberFeatureData)structMemDef;

```

```

        Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " +
PrimDef.PrimaryMemberType);
        facePlaneIntrSecDef = (PrimaryMemberFacePlaneIntersectionFeatureData)PrimDef;
        facePlaneIntrSecDef.MergeTangentMembers = false;

        // Select parameter face and intersecting plane
        stat = modDocExt.SelectByRay(-0.00017443058891331d, 0.0116544186780629d, -0.000426704146832435d, -0.400036026779314d,
-0.515038074910018d, -0.758094294050287d, 0.000189520697225961d, 2, false, 0, 0);
        faces[0] = swSelMgr.GetSelectedObject6(1, -1);
        stat = modDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
        IntSecPlane[0] = swSelMgr.GetSelectedObject6(1, -1);
        DispatchWrapper[] dArray;
        DispatchWrapper[] eArray;
        DispatchWrapper[] fArray;
        dArray = ObjectArrayToDispatchWrapperArray(faces);
        stat = facePlaneIntrSecDef.SetParameterFaces(dArray);
        Debug.Print("Member's parameter face set successfully: " + stat);
        eArray = ObjectArrayToDispatchWrapperArray(IntSecPlane);
        stat = facePlaneIntrSecDef.SetIntersectingPlanesAndFaces(eArray);
        Debug.Print("Member's intersecting plane set successfully: " + stat);
        memData[0] = structMemDef;
        fArray = ObjectArrayToDispatchWrapperArray(memData);

        // Create Structure System1 in the FeatureManager design tree
        structSys = modDocExt.CreateStructureSystem(fArray, null);
        modDoc.ViewZoomtofit2();
        Debugger.Break();
        Debug.Print("");
        Debug.Print("Edit Structure System1...");
        structSysModDef = (StructureSystemFolder)structSys.GetSpecificFeature2();
        Debug.Print("Number of structure system profile groups: " + structSysModDef.GetProfileGroupFoldersCount());
        outProfiles = (object[])structSysModDef.GetProfileGroupFolders();
        if (outProfiles.Length > 0)
        {
            var loopTo = outProfiles.GetUpperBound(0);
            for (I = 0; I <= loopTo; I++)
            {
                profileGrpFeat = (Feature)outProfiles[(int)I];
                profileGrp = (ProfileGroupFolder)profileGrpFeat.GetSpecificFeature2();
                if (profileGrp is object)
                {
                    Debug.Print("Number of members inside group: " + profileGrp.GetStructureSystemMemberCount());
                    memberArray = (object[])profileGrp.GetStructureSystemMembers();
                    if (memberArray.Length > 0)
                    {
                        var loopTo1 = memberArray.GetUpperBound(0);
                        for (J = 0; J <= loopTo1; J++)
                        {
                            MemberData = (StructureSystemMemberFeatureData)memberArray[(int)J];
                            break;
                        }
                    }
                }
            }
        }

        memToChange = MemberData;
        feat = memToChange.GetFeature();
        Debug.Print("Change member extensions in directions 1 and 2...");
        memToChange.StartEndExtendD1 = 0.1d;
        memToChange.StartEndExtendD2 = 0.2d;
        faceIntrMemberToChange = (PrimaryMemberFacePlaneIntersectionFeatureData)memToChange;

```

```
modDoc.ClearSelection2(true);
stat = modDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
newIntSecPlane[0] = swSelMgr.GetSelectedObject6(1, -1);
dArray = ObjectArrayToDispatchWrapperArray(newIntSecPlane);
stat = faceIntrMemberToChange.SetIntersectingPlanesAndFaces(dArray);
Debug.Print("Intersecting plane successfully changed: " + stat);
stat = feat.ModifyDefinition(MemberData, modDoc, null);
}
```

```
// The SldWorks swApp variable is pre-assigned for you.
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Create Face Plane Intersection Structure System Member (VB.NET)

This example shows how to create a structure system member along the intersection of a face and a plane.

```
'=====
'Preconditions:
'1. Ensure the specified part template exists.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
' 1. Creates Boss-Extrude1 and Plane1 in a new part.
' 2. Selects a parameter face on the extrusion.
' 3. Selects Plane1.
' 4. Creates a face plane intersection member along the intersection of the selected face and plane.
' 5. Creates a Structure System1.
' 6. Inspect the member on Plane1 in the graphics area.
' 7. Press F5 to edit the structure system.
' 8. Gets all of the structure system members in all of the profile group folders.
' 9. Changes the member extension lengths in directions 1 and 2.
'10. Selects the Right Plane to intersect the parameter face.
'11. Modifies the structure system.
'12. Inspect the member move to the Right Plane in the graphics area.
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Diagnostics
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim modDoc As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim modDocExt As ModelDocExtension
    Dim structMemDef As StructureSystemMemberFeatureData
    Dim profDef As StructureSystemMemberProfile
    Dim PrimDef As PrimaryStructuralMemberFeatureData
    Dim facePlaneIntrSecDef As PrimaryMemberFacePlaneIntersectionFeatureData
    Dim faces(0) As Object
    Dim stat As Boolean
    Dim IntSecPlane(0) As Object
    Dim memData(0) As StructureSystemMemberFeatureData
    Dim memDataArray() As Object
    Dim structSys As Feature
    Dim structSysModDef As StructureSystemFolder
    Dim outProfiles() As Object
    Dim MemberData As StructureSystemMemberFeatureData
    Dim memberArray() As Object
    Dim profileGrpFeat As Feature
    Dim profileGrp As ProfileGroupFolder
    Dim memToChange As StructureSystemMemberFeatureData
    Dim faceIntrMemberToChange As PrimaryMemberFacePlaneIntersectionFeatureData
    Dim newIntSecPlane(0) As Object
    Dim feat As Feature
    Dim i As Integer
    Dim j As Integer
```

```

Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As DispatchWrapper
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

Sub main()

    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    modDoc = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0, swSheetWidth,
swSheetHeight)
    Dim swPart As PartDoc
    swPart = modDoc
    modDoc = swApp.ActiveDoc

    modDocExt = modDoc.Extension
    swSelMgr = modDoc.SelectionManager

    stat = modDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.SketchManager.InsertSketch(True)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    stat = modDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Dim vSkLines() As Object
    vSkLines = modDoc.SketchManager.CreateCornerRectangle(-0.0168788132580097, 0.0116544186781496, 0, 0.0139317188796271,
-0.00736773594595662, 0)

    modDoc.ShowNamedView2("**Trimetric", 8)
    modDoc.ViewZoomtofit2()
    Dim myFeature As Feature
    myFeature = modDoc.FeatureManager.FeatureExtrusion2(False, False, False, 0, 0, 0.2, 0.2, False, False, False, False,
0.0174532925199433, 0.0174532925199433, False, False, False, True, True, True, 0, 0, False)
    swSelMgr.EnableContourSelection = False
    stat = modDocExt.SelectByRay(0.199999999999932, 0.00280649656951937, 0.000880259028576802, -0.400036026779314,
-0.515038074910018, -0.758094294050287, 0.00111139990390966, 2, False, 0, 0)
    Dim myRefPlane As RefPlane
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(4, 0, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)

    'Configure a structure system face plane intersection member
    structMemDef = modDocExt.CreateStructureSystemMemberData(3)
    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

    structMemDef.StartEndExtendD1 = 0.4
    Debug.Print("Extend length in direction 1: " & structMemDef.StartEndExtendD1)
    structMemDef.StartEndExtendD2 = 0.15
    Debug.Print("Extend length in direction 2: " & structMemDef.StartEndExtendD2)

    profDef = structMemDef.MemberProfile

```

```

profDef.ProfileStandard = "ansi inch"
profDef.ProfileType = "c channel"
profDef.ProfileSize = "3 x 5"

PrimDef = structMemDef
Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
PrimDef.PrimaryMemberType)

facePlaneIntrSecDef = PrimDef

facePlaneIntrSecDef.MergeTangentMembers = False

'Select parameter face and intersecting plane
stat = modDocExt.SelectByRay(-0.00017443058891331, 0.0116544186780629, -0.000426704146832435, -0.400036026779314,
-0.515038074910018, -0.758094294050287, 0.000189520697225961, 2, False, 0, 0)
faces(0) = swSelMgr.GetSelectedObject6(1, -1)
stat = modDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
IntSecPlane(0) = swSelMgr.GetSelectedObject6(1, -1)

Dim dArray() As DispatchWrapper
Dim eArray() As DispatchWrapper
Dim fArray() As DispatchWrapper

dArray = ObjectArrayToDispatchWrapperArray(faces)

stat = facePlaneIntrSecDef.SetParameterFaces(dArray)
Debug.Print("Member's parameter face set successfully: " & stat)

eArray = ObjectArrayToDispatchWrapperArray(IntSecPlane)

stat = facePlaneIntrSecDef.SetIntersectingPlanesAndFaces(eArray)
Debug.Print("Member's intersecting plane set successfully: " & stat)

memData(0) = structMemDef

fArray = ObjectArrayToDispatchWrapperArray(memData)

'Create Structure System1 in the FeatureManager design tree
structSys = modDocExt.CreateStructureSystem(fArray, Nothing)

modDoc.ViewZoomtofit2()

Stop

Debug.Print("")
Debug.Print("Edit Structure System1...")

structSysModDef = structSys.GetSpecificFeature2()
Debug.Print("Number of structure system profile groups: " & structSysModDef.GetProfileGroupFoldersCount)

outProfiles = structSysModDef.GetProfileGroupFolders()

If (outProfiles.Length > 0) Then
    For I = 0 To UBound(outProfiles)
        profileGrpFeat = outProfiles(I)
        profileGrp = profileGrpFeat.GetSpecificFeature2
        If Not profileGrp Is Nothing Then
            Debug.Print("Number of members inside group: " & profileGrp.GetStructureSystemMemberCount)
            memberArray = profileGrp.GetStructureSystemMembers()
            If (memberArray.Length > 0) Then
                For j = 0 To UBound(memberArray)

```

```

        MemberData = memberArray(j)
    Exit For
Next
End If
End If
Next
End If

memToChange = MemberData
feat = memToChange.GetFeature()

Debug.Print("Change member extensions in directions 1 and 2...")
memToChange.StartEndExtendD1 = 0.1
memToChange.StartEndExtendD2 = 0.2

faceIntrMemberToChange = memToChange

modDoc.ClearSelection2(True)

stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
newIntSecPlane(0) = swSelMgr.GetSelectedObject6(1, -1)
dArray = ObjectArrayToDispatchWrapperArray(newIntSecPlane)
stat = faceIntrMemberToChange.SetIntersectingPlanesAndFaces(dArray)
Debug.Print("Intersecting plane successfully changed: " & stat)

stat = feat.ModifyDefinition(MemberData, modDoc, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Feature with Invalid Geometry Example (VBA)

This example shows how to prompt the user whether to create a feature that has invalid geometry.

```
'-----  
'  
' Preconditions: None  
'  
' Postconditions: If user clicks Yes, then create a feature that has invalid  
'                 geometry; if No, then do not create a feature that has invalid  
'                 geometry.  
'  
'-----  
  
Option Explicit  
Sub main()  
    Dim swApp           As SldWorks.SldWorks  
    Dim nResponse       As Integer  
    Dim nStatus         As Long  
    Dim bRet            As Boolean  
  
    Set swApp = Application.SldWorks  
  
    nResponse = MsgBox("Create feature that has invalid geometry?", vbYesNo)  
    If nResponse = vbYes Then  
        bRet = swApp.AllowFailedFeatureCreation(True)  
    Else  
        bRet = swApp.AllowFailedFeatureCreation(False)  
    End If  
End Sub  
'-----
```

Create Fill Pattern Example (C#)

This example shows how to create a Fill Pattern feature and get its data.

```
//-----
// Preconditions: Ensure the specified part exists.
//
// Postconditions:
// 1. Extrudes a hole in the middle of the block.
// 2. Creates a circular fill pattern of the specified hole.
// 3. Inspect the graphics area and the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateFillPattern_CSharp
{
    partial class SolidWorksMacro
    {
        private Feature myFeature;
        private Feature swFeat;
        private FeatureManager swFeatMgr;
        private ModelDoc2 Part;
        private FillPatternFeatureData swFeatData;
        private bool boolstatus;

        public void Main()
        {
            int longstatus = 0;
            int longwarnings = 0;
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\block20.sldprt",
            swApp.ActivateDoc2("block20.sldprt", false, longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("", "", -0.0032, 0.0418, 0.0, false, 0, null, 0);
            myFeature = Part.FeatureManager.SimpleHole2(0.0054, true, false, false, 1, 0, 0.0254, 0.0254, false, false, false, false,
            ((SelectionMgr)(Part.SelectionManager)).EnableContourSelection = false;
            Part.ActivateSelectedFeature();

            boolstatus = Part.Extension.SelectByID2("Hole1", "BODYFEATURE", 0, 0, 0, false, 4, null, 0);
            boolstatus = Part.Extension.SelectByRay(0.00106119575008097, 0.039624, 0.0493456023787655, -0.400036026779312, -0.515038
            boolstatus = Part.Extension.SelectByRay(0.0636836165047043, 0.0396239999998329, -0.000633171793538168, -0.40003602677931

            swFeatMgr = Part.FeatureManager;

            swFeatData = (FillPatternFeatureData)swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmFillPattern);
            swFeatData.CreateSeedCutPolygonSides = 0;
            swFeatData.CreateSeedCutType = 0;
            swFeatData.Diagonal = 0.008;
            swFeatData.Diameter = 0.008;
            swFeatData.Dimension = 0.00565685424949238;
            swFeatData.FeaturesToPatternType = 0;
            swFeatData.GeometryPattern = false;
            swFeatData.InnerRadius = 0.00380422606518061;
            swFeatData.InstanceSpacing = 0.0170240490494193;
            swFeatData.LayoutSpacingType = 1;
            swFeatData.LoopSpacing = 0.0170240490494193;
            swFeatData.Margins = 0;
            swFeatData.NoOfInstances = 10;
            swFeatData.OuterRadius = 0.004;
            swFeatData.PatternLayoutPolygonSides = 0;
            swFeatData.PatternLayoutType = 0;
            swFeatData.Rotation = 0;
            swFeatData.SeedCutFlipShapeDirection = false;
            swFeatData.StaggerAngle = 1.0471975511966;
            swFeat = swFeatMgr.CreateFeature(swFeatData);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Flyouts in the CommandManager Example (C#)

This example shows how to use an add-in to create flyouts in:

- CommandManager
- context-sensitive menus of selected faces

```
//-----
// Preconditions:
// 1. Verify that the SOLIDWORKS API SDK add-in templates for your version
// of Microsoft Visual Studio are installed.
// 2. In Microsoft Visual Studio, create a C# project using the
// SwCSharpAddin template.
// 3. Name the project FlyoutAddin.
// 4. Copy and paste SwAddin into SwAddin.cs of your project.
// 5. Click Project > FlyoutAddin Properties > Debug.
// 6. Select Start external program and type the pathname of your
// SOLIDWORKS executable.
// 7. Replace the text shown for the icons and mainIcons array elements
// with the pathnames of your image files.
// 8. Press F5 to start debugging this add-in.
//
// Postconditions:
// 1. In SOLIDWORKS, click Tools > C# Add-in > CreateCube.
// 2. Click Dynamic Flyout in the CommandManager and click FlyoutCommand 2.
// 3. Inspect the Immediate window.
// 4. Click a face of the cube.
// 5. Click the flyout button in the face's context-sensitive menu and
// click FlyoutCommand 1.
// 6. Inspect the Immediate window.
//-----

//SwAddin

using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Collections.Generic;
using System.Diagnostics;

namespace FlyoutAddin
{
    /// <summary>
    /// This C# add-in shows how to create a flyout menu in both the CommandManager
    /// and the context-sensitive menus of selected faces.
    /// </summary>
    [Guid("cc6bb106-bf87-49e0-b0fb-dcb49eab0bbe"), ComVisible(true)]
    [SwAddin(
        Description = "A flyout menu appears on the toolbar and on the context-sensitive menus of selected faces.",
        Title = "C# Add-in",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        ISldWorks iSwApp = null;
        ICommandManager iCmdMgr = null;
        ICommandGroup cmdGroup;
        IFlyoutGroup flyoutGroup;
        int addinID = 0;
        BitmapHandler iBmp;

        public const int mainCmdGroupID = 5;
        public const int mainItemID1 = 0;
        public const int mainItemID2 = 1;
        public const int mainItemID3 = 2;
        public const int flyoutGroupID = 91;

        string[] mainIcons = new string[3];
        string[] icons = new string[3];

        #region Event Handler Variables
        Hashtable openDocs = new Hashtable();
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr = null;
        #endregion

        #region Property Manager Variables
        UserPMPage ppage = null;
        #endregion

        // Public Properties
        public ISldWorks SwApp
        {
            get { return iSwApp; }
        }
        public ICommandManager CmdMgr
        {
            get { return iCmdMgr; }
        }

        public Hashtable OpenDocs
        {
            get { return openDocs; }
        }

        #endregion

        #region SOLIDWORKS Registration
        [ComRegisterFunctionAttribute]
        public static void RegisterFunction(Type t)
        {

```

```

{
    #region Get Custom Attribute: SwAddinAttribute
    SwAddinAttribute SwAttr = null;
    Type type = typeof(SwAddin);

    foreach (System.Attribute attr in type.GetCustomAttributes(false))
    {
        if (attr is SwAddinAttribute)
        {
            SwAttr = attr as SwAddinAttribute;
            break;
        }
    }

    #endregion

    try
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\{" + t.GUID.ToString() + "}";
        Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
        addinkey.SetValue(null, 0);

        addinkey.SetValue("Description", SwAttr.Description);
        addinkey.SetValue("Title", SwAttr.Title);

        keyname = "Software\\SOLIDWORKS\\AddInsStartup\\{" + t.GUID.ToString() + "}";
        addinkey = hkcu.CreateSubKey(keyname);
        addinkey.SetValue(null, Convert.ToInt32(SwAttr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem registering this dll: SwAttr is null. \n\" + nl.Message + "\");
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: SwAttr is null.\n\" + nl.Message + "\");
    }

    catch (System.Exception e)
    {
        Console.WriteLine(e.Message);

        System.Windows.Forms.MessageBox.Show("There was a problem registering the function: \n\" + e.Message + "\");
    }
}

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction(Type t)
{
    try
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\{" + t.GUID.ToString() + "}";
        hklm.DeleteSubKey(keyname);

        keyname = "Software\\SOLIDWORKS\\AddInsStartup\\{" + t.GUID.ToString() + "}";
        hkcu.DeleteSubKey(keyname);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + nl.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\" + nl.Message + "\");
    }
    catch (System.Exception e)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + e.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\" + e.Message + "\");
    }
}

#endregion

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    //Set up callbacks
    iSwApp.SetAddinCallbackInfo2(0, this, addinID);

    #region Set up the CommandManager
    iCmdMgr = iSwApp.GetCommandManager(cookie);
    AddCommandMgr();
    #endregion

    #region Set up the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    #region Set up PropertyManager page
    AddPMP();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    RemoveCommandMgr();
    RemovePMP();
    DetachEventHandlers();

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iCmdMgr);
    iCmdMgr = null;
    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSwApp);
    iSwApp = null;
    // The add-in must call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect();
    GC.WaitForPendingFinalizers();

    GC.Collect();
}

```

```

        GC.WaitForPendingFinalizers();
    }
    return true;
}
#endregion

#region UI Methods
public void updateBtns()
{
    flyoutGroup = iCmdMgr.GetFlyoutGroup(91);

    flyoutGroup.RemoveAllCommandItems();

    flyoutGroup.AddCommandItem("FlyoutCommand 1", "FlyoutCommand 1", 0, "FlyoutCommandItem1", "FlyoutEnableCommandItem1");
    flyoutGroup.AddCommandItem("FlyoutCommand 2", "FlyoutCommand 2", 0, "FlyoutCommandItem2", "FlyoutEnableCommandItem2");

    flyoutGroup.FlyoutType = (int)swCommandFlyoutStyle_e.swCommandFlyoutStyle_Simple;

    IFlyoutGroup fogrp;
    fogrp = iCmdMgr.GetFlyoutGroup(91);
    Debug.Print(" CmdID: " + fogrp.CmdID);
    Debug.Print(" Button count: " + fogrp.ButtonCount);
    Debug.Print(" Flyout type: " + fogrp.FlyoutType);
    mainIcons = (string[])fogrp.MainIconList;
    Debug.Print(" Small button: " + mainIcons[0]);
    Debug.Print(" Medium button: " + mainIcons[1]);
    Debug.Print(" Large button: " + mainIcons[2]);
    icons = (string[])fogrp.IconList;
    Debug.Print(" Small toolbar button: " + icons[0]);
    Debug.Print(" Medium toolbar button: " + icons[1]);
    Debug.Print(" Large toolbar button: " + icons[2]);
}

public void AddCommandMgr()
{
    if (iBmp == null)
        iBmp = new BitmapHandler();
    Assembly thisAssembly;
    int cmdIndex0, cmdIndex1;
    string Title = "C# Add-in", ToolTip = "Flyout demo";

    int[] docTypes = new int[] { (int)swDocumentTypes_e.swDocASSEMBLY,
        (int)swDocumentTypes_e.swDocDRAWING,
        (int)swDocumentTypes_e.swDocPART };

    thisAssembly = System.Reflection.Assembly.GetAssembly(this.GetType());

    int cmdGroupErr = 0;
    bool ignorePrevious = false;

    object registryIDs;
    // Get the ID information stored in the registry
    bool getDataResult = iCmdMgr.GetGroupDataFromRegistry(mainCmdGroupID, out registryIDs);

    int[] knownIDs = new int[2] { mainItemID1, mainItemID2 };

    if (getDataResult)
    {
        if (!CompareIDs((int[])registryIDs, knownIDs)) // If the IDs don't match, reset the CommandGroup
        {
            ignorePrevious = true;
        }
    }

    int smallImage = 0;
    int mediumImage = 0;
    int largeImage = 0;
    int imageSizeToUse = 0;

    imageSizeToUse = SwApp.GetImageSize(out smallImage, out mediumImage, out largeImage);

    Debug.Print("Image sizes:");
    Debug.Print(" Default PropertyManager page and menu image size based on DPI setting: " + imageSizeToUse);
    Debug.Print(" Small image size based on DPI setting: " + smallImage);
    Debug.Print(" Medium image size based on DPI setting: " + mediumImage);
    Debug.Print(" Large image size based on DPI setting: " + largeImage);

    cmdGroup = iCmdMgr.CreateCommandGroup2(mainCmdGroupID, Title, ToolTip, "", -1, ignorePrevious, ref cmdGroupErr);

    icons[0] = "Pathname_to_toolbar_nxn_image";
    icons[1] = "Pathname_to_toolbar_nnxnn_image";
    icons[2] = "Pathname_to_toolbar_nnnxnnn_image";
    mainIcons[0] = "Pathname_to_nxn_image";
    mainIcons[1] = "Pathname_to_nnxnn_image";
    mainIcons[2] = "Pathname_to_nnnxnnn_image";

    cmdGroup.IconList = icons;
    cmdGroup.MainIconList = mainIcons;

    int menuToolbarOption = (int)(swCommandItemType_e.swMenuItem | swCommandItemType_e.swToolbarItem);
    cmdIndex0 = cmdGroup.AddCommandItem2("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", mainItemID1, menuToolbarOption);
    cmdIndex1 = cmdGroup.AddCommandItem2("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP", "EnablePMP", mainItemID2, menuToolbarOption);

    cmdGroup.HasToolbar = true;
    cmdGroup.HasMenu = true;
    cmdGroup.Activate();

    bool bResult;

    flyoutGroup = iCmdMgr.CreateFlyoutGroup2(flyoutGroupID, "Dynamic Flyout", "Dynamic Flyout", "Flyout Hint",
        mainIcons, icons, "FlyoutCallback", "FlyoutEnable");

    // Add the FlyoutGroup to the context-sensitive menus of faces in parts
    bResult = flyoutGroup.AddContextMenuFlyout((int)swDocumentTypes_e.swDocPART, (int)swSelectType_e.swSelFACES);
    Debug.Print("Context-sensitive menu flyout created for faces in parts: " + bResult.ToString());

    // Get the total number of FlyoutGroups in CommandManager
    Debug.Print("Number of FlyoutGroups is " + iCmdMgr.NumberOfFlyoutGroups);

    // Get the FlyoutGroups
    object[] objGroups;
    objGroups = (object[])iCmdMgr.GetFlyoutGroups();
    Debug.Print("Find all FlyoutGroups in CommandManager:");
    int i;

```

```

for (i = 0; i <= objGroups.GetUpperBound(0); i++)
{
    Debug.Print("FlyoutGroup found");
}

// Get a FlyoutGroup by its user-defined ID
IFlyoutGroup fogrp;
fogrp = iCmdMgr.GetFlyoutGroup(91);
Debug.Print(" CmdID: " + fogrp.CmdID);
Debug.Print(" Button count: " + fogrp.ButtonCount);
Debug.Print(" Flyout Type: " + fogrp.FlyoutType);
mainIcons = (string[])fogrp.MainIconList;
Debug.Print(" Small button: " + mainIcons[0]);
Debug.Print(" Medium button: " + mainIcons[1]);
Debug.Print(" Large button: " + mainIcons[2]);
icons = (string[])fogrp.IconList;
Debug.Print(" Small toolbar button: " + icons[0]);
Debug.Print(" Medium toolbar button: " + icons[1]);
Debug.Print(" Large toolbar button: " + icons[2]);

foreach (int type in docTypes)
{
    CommandTab cmdTab;

    cmdTab = iCmdMgr.GetCommandTab(type, Title);

    if (cmdTab != null && !getDataResult ||
        ignorePrevious) // If tab exists, but we have ignored the registry info (or changed CommandGroup ID), re-
        create the tab; otherwise the ids won't match and the tab will be blank
    {
        bool res = iCmdMgr.RemoveCommandTab(cmdTab);
        cmdTab = null;
    }

    // If cmdTab is null, must be first load (possibly after reset), add the commands to the tabs
    if (cmdTab == null)
    {
        cmdTab = iCmdMgr.AddCommandTab(type, Title);

        CommandTabBox cmdBox = cmdTab.AddCommandTabBox();

        int[] cmdIDs = new int[3];
        int[] TextType = new int[3];

        cmdIDs[0] = cmdGroup.get_CommandID(cmdIndex0);
        TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[1] = cmdGroup.get_CommandID(cmdIndex1);
        TextType[1] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

        cmdIDs[2] = cmdGroup.ToolbarID;
        TextType[2] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal |
        (int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

        bResult = cmdBox.AddCommands(cmdIDs, TextType);

        CommandTabBox cmdBox1 = cmdTab.AddCommandTabBox();
        cmdIDs = new int[1];
        TextType = new int[1];

        cmdIDs[0] = flyoutGroup.CmdID;
        TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow |
        (int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

        bResult = cmdBox1.AddCommands(cmdIDs, TextType);

        cmdTab.AddSeparator(cmdBox1, cmdIDs[0]);
    }
}

thisAssembly = null;
}

public void RemoveCommandMgr()
{
    iBmp.Dispose();

    iCmdMgr.RemoveCommandGroup(mainCmdGroupID);
    iCmdMgr.RemoveFlyoutGroup(flyoutGroupID);
}

public bool CompareIDs(int[] storedIDs, int[] addinIDs)
{
    List<int> storedList = new List<int>(storedIDs);
    List<int> addinList = new List<int>(addinIDs);

    addinList.Sort();
    storedList.Sort();

    if (addinList.Count != storedList.Count)
    {
        return false;
    }
    else
    {
        for (int i = 0; i < addinList.Count; i++)
        {
            if (addinList[i] != storedList[i])
            {
                return false;
            }
        }
    }
    return true;
}

public Boolean AddPMP()
{
    ppage = new UserPMPPage(this);
    return true;
}

```

```

public Boolean RemovePMP()
{
    ppage = null;
    return true;
}

#endregion

#region UI Callbacks
public void CreateCube()
{
    // Make sure a part open is open
    string partTemplate = iSwApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
    if ((partTemplate != null) && (partTemplate != ""))
    {
        IModelDoc2 modDoc = (IModelDoc2)iSwApp.NewDocument(partTemplate, (int)swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0);
        ISketchManager sketchMgr = modDoc.SketchManager;
        sketchMgr.InsertSketch(true);
        sketchMgr.CreateCornerRectangle(0, 0, 0, .1, .1, .1);
        // Extrude the sketch
        IFeatureManager featMan = modDoc.FeatureManager;
        featMan.FeatureExtrusion3(true,
            false, false,
            (int)swEndConditions_e.swEndCondBlind, (int)swEndConditions_e.swEndCondBlind,
            0.1, 0.0,
            false, false,
            false, false,
            0.0, 0.0,
            false, false,
            false, false,
            false,
            false, true, (int)swStartConditions_e.swStartSketchPlane, 0.0, false);
    }
    else
    {
        System.Windows.Forms.MessageBox.Show("There is no part template available. Please check your options and make sure there is a part template selected, or select a new part template.");
    }
}

public void ShowPMP()
{
    if (ppage != null)
        ppage.Show();
}

public int EnablePMP()
{
    if (iSwApp.ActiveDoc != null)
        return 1;
    else
        return 0;
}

public void FlyoutCallback()
{
    updateBtns();
}

public int FlyoutEnable()
{
    // Enable the flyout only if CommandGroup buttons are enabled
    if (cmdGroup.HasEnabledButton)
        return 1;
    else
        return 0;
}

public void FlyoutCommandItem1() { Debug.Print("Flyout command 1 called"); }

public int FlyoutEnableCommandItem1() { return 1; }

public void FlyoutCommandItem2() { Debug.Print("Flyout command 2 called"); }

public int FlyoutEnableCommandItem2() { return 1; }

#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    // Listen for events on all currently open docs
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        return true;
    }
    catch (Exception e)
    {
    }
}

```

```

        {
            Console.WriteLine(e.Message);
            return false;
        }
    }

    public void AttachEventsToAllDocuments()
    {
        ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
        while (modDoc != null)
        {
            if (!openDocs.Contains(modDoc))
            {
                AttachModelDocEventHandler(modDoc);
            }
            modDoc = (ModelDoc2)modDoc.GetNext();
        }
    }

    public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
    {
        if (modDoc == null)
            return false;

        DocumentEventHandler docHandler = null;

        if (!openDocs.Contains(modDoc))
        {
            switch (modDoc.GetType())
            {
                case (int)swDocumentTypes_e.swDocPART:
                {
                    docHandler = new PartEventHandler(modDoc, this);
                    break;
                }
                case (int)swDocumentTypes_e.swDocASSEMBLY:
                {
                    docHandler = new AssemblyEventHandler(modDoc, this);
                    break;
                }
                case (int)swDocumentTypes_e.swDocDRAWING:
                {
                    docHandler = new DrawingEventHandler(modDoc, this);
                    break;
                }
                default:
                {
                    return false; // Unsupported document type
                }
            }
            docHandler.AttachEventHandlers();
            openDocs.Add(modDoc, docHandler);
        }
        return true;
    }

    public bool DetachModelEventHandler(ModelDoc2 modDoc)
    {
        DocumentEventHandler docHandler;
        docHandler = (DocumentEventHandler)openDocs[modDoc];
        openDocs.Remove(modDoc);
        modDoc = null;
        docHandler = null;
        return true;
    }

    public bool DetachEventHandlers()
    {
        DetachSwEvents();

        // Close events on all currently open docs
        DocumentEventHandler docHandler;
        int numKeys = openDocs.Count;
        object[] keys = new Object[numKeys];

        // Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0);
        foreach (ModelDoc2 key in keys)
        {
            docHandler = (DocumentEventHandler)openDocs[key];
            docHandler.DetachEventHandlers(); // This also removes the pair from the hash
            docHandler = null;
        }
        return true;
    }
}
#endregion

#region Event Handlers
// Events
public int OnDocChange()
{
    return 0;
}

public int OnDocLoad(string docTitle, string docPath)
{
    return 0;
}

int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnFileNew(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnModelChange()
{
    return 0;
}
}
#endregion
}

```


Create Flyouts in the CommandManager Example (VB.NET)

This example shows how to use an add-in to create flyouts in:

- CommandManager
- context-sensitive menus of selected faces

```

-----
Preconditions:
1. Verify that the SOLIDWORKS API SDK add-in templates for your version
  of Microsoft Visual Studio are installed.
2. In Microsoft Visual Studio, create a Visual Basic project using the
  SwVBAddin template.
3. Name the project SwVBAddin1.
4. Copy and paste SwAddin into SwAddin.vb of your project.
5. Click Project > SwVBAddin1 Properties > Debug.
6. Select Start external program and type the pathname to your
  SOLIDWORKS executable.
7. Replace the text shown for the icons and mainIcons array elements
  with the pathnames to your image files.
8. Press F5 to start debugging this add-in.
-----
Postconditions:
1. In SOLIDWORKS, click Tools > VB.NET Add-in > CreateCube.
2. Click Dynamic Flyout in the CommandManager and click FlyoutCommand 2.
3. Inspect the Immediate window.
4. Click a face of the cube.
5. Click the flyout button in the face's context-sensitive menu and
  click FlyoutCommand 1.
6. Inspect the Immediate window.
-----

' SwAddin

Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SolidWorksTools
Imports SolidWorksTools.File

Imports System.Collections.Generic
Imports System.Diagnostics

' This VB.NET add-in shows how to create a flyout menu in both the CommandManager
' and the context-sensitive menus of selected faces

<Guid("88bf8dab-7623-4846-9a76-c1f64a2ebeb8")> _
<ComVisible(True)> _
<SwAddin(
    Description:="A flyout menu appears on the toolbar and on the context-sensitive menus of selected faces.", _
    Title:="VB.NET Add-in", _
    LoadAtStartup:=True _
)>
Public Class SwAddin
    Implements SolidWorks.Interop.swpublished.SwAddin

#Region "Local Variables"
    Dim WithEvents iSwApp As SldWorks
    Dim iCmdMgr As ICommandManager
    Dim addinID As Integer
    Dim openDocs As Hashtable
    Dim SwEventPtr As SldWorks
    Dim ppage As UserPMPPage
    Dim iBmp As BitmapHandler
    Dim cmdGroup As ICommandGroup

    Public Const mainCmdGroupID As Integer = 0
    Public Const mainItemID1 As Integer = 0
    Public Const mainItemID2 As Integer = 1
    Public Const flyoutGroupID As Integer = 91

    ' Public Properties
    ReadOnly Property SwApp() As SldWorks
        Get
            Return iSwApp
        End Get
    End Property

    ReadOnly Property CmdMgr() As ICommandManager
        Get
            Return iCmdMgr
        End Get
    End Property

    ReadOnly Property OpenDocumentsTable() As Hashtable
        Get
            Return openDocs
        End Get
    End Property
#End Region

#Region "SOLIDWORKS Registration"

    <ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)
        ' Get Custom Attribute: SwAddinAttribute
        Dim attributes() As Object
        Dim SwAttr As SwAddinAttribute = Nothing

        attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

        If attributes.Length > 0 Then
            SwAttr = DirectCast(attributes(0), SwAddinAttribute)
        End If
    End Sub
End Sub

```

```

End If
Try
    Dim hkml As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
    Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

    Dim keyname As String = "SOFTWARE\SOLIDWORKS\AddIns\{" + t.GUID.ToString() + "}"
    Dim addinkey As Microsoft.Win32.RegistryKey = hkml.CreateSubKey(keyname)
    addinkey.SetValue(Nothing, 0)
    addinkey.SetValue("Description", Swattr.Description)
    addinkey.SetValue("Title", Swattr.Title)

    keyname = "Software\SOLIDWORKS\AddInsStartup\{" + t.GUID.ToString() + "}"
    addinkey = hkcu.CreateSubKey(keyname)
    addinkey.SetValue(Nothing, Swattr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
Catch nl As System.NullReferenceException
    Console.WriteLine("There was a problem registering this dll: Swattr is null.\n " & nl.Message)
    System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: Swattr is null.\n " & nl.Message)
Catch e As System.Exception
    Console.WriteLine("There was a problem registering this dll: " & e.Message)
    System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: " & e.Message)
End Try
End Sub

<ComUnregisterFunction(> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Try
        Dim hkml As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SOLIDWORKS\AddIns\{" + t.GUID.ToString() + "}"
        hkml.DeleteSubKey(keyname)

        keyname = "Software\SOLIDWORKS\AddInsStartup\{" + t.GUID.ToString() + "}"
        hkcu.DeleteSubKey(keyname)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem unregistering this dll: Swattr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: Swattr is null.\n " & nl.Message)
    Catch e As System.Exception
        Console.WriteLine("There was a problem unregistering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: " & e.Message)
    End Try
End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSWApp = ThisSW
    addinID = Cookie

    ' Set up callbacks
    iSWApp.SetAddinCallbackInfo2(0, Me, addinID)

    ' Set up the CommandManager
    iCmdMgr = iSWApp.GetCommandManager(Cookie)
    AddCommandMgr()

    ' Set up the Event Handlers
    SwEventPtr = iSWApp
    openDocs = New Hashtable
    AttachEventHandlers()

    ' Set up PropertyManager page
    AddPMP()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW

    RemoveCommandMgr()
    RemovePMP()
    DetachEventHandlers()

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iCmdMgr)
    iCmdMgr = Nothing
    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSWApp)
    iSWApp = Nothing
    ' The add-in must call GC.Collect() here to retrieve all managed code pointers
    GC.Collect()
    GC.WaitForPendingFinalizers()

    GC.Collect()
    GC.WaitForPendingFinalizers()

    DisconnectFromSW = True
End Function
#End Region

#Region "UI Methods"
Public Sub AddCommandMgr()

    If iBmp Is Nothing Then
        iBmp = New BitmapHandler()
    End If

    Dim thisAssembly As Assembly

    Dim cmdIndex0 As Integer, cmdIndex1 As Integer
    Dim Title As String = "VB.NET Add-in"
    Dim ToolTip As String = "Flyout demo"

    Dim docTypes() As Integer = {swDocumentTypes_e.swDocASSEMBLY, _
        swDocumentTypes_e.swDocDRAWING, _
        swDocumentTypes_e.swDocPART}

    thisAssembly = System.Reflection.Assembly.GetAssembly(Me.GetType())

    Dim cmdGroupErr As Integer = 0
    Dim ignorePrevious As Boolean = False

    Dim registryIDs As Object = Nothing
    Dim getResult As Boolean = iCmdMgr.GetGroupDataFromRegistry(mainCmdGroupID, registryIDs)

    Dim knownIDs As Integer() = New Integer(1) {mainItemID1, mainItemID2}

    If getResult Then
        If Not CompareIDs(registryIDs, knownIDs) Then ' If the IDs don't match, reset the CommandGroup

```

```

        ignorePrevious = True
    End If
End If

Dim smallImage As Integer
Dim mediumImage As Integer
Dim largeImage As Integer
Dim imageSizeToUse As Integer

imageSizeToUse = SwApp.GetImageSize(smallImage, mediumImage, largeImage)

Debug.Print("Image sizes:")
Debug.Print(" Default PropertyManager page and menu icon size based on DPI setting: " & imageSizeToUse)
Debug.Print(" Small image size based on DPI setting: " & smallImage)
Debug.Print(" Medium image size based on DPI setting: " & mediumImage)
Debug.Print(" Large image size based on DPI setting: " & largeImage)

cmdGroup = iCmdMgr.CreateCommandGroup2(mainCmdGroupID, Title, ToolTip, "", -1, ignorePrevious, cmdGroupErr)
If cmdGroup Is Nothing Or thisAssembly Is Nothing Then
    Throw New NullReferenceException()
End If

Dim mainIcons(2) As String
Dim icons(2) As String
icons(0) = "Pathname_to_toolbar_nxn_image"
icons(1) = "Pathname_to_toolbar_nnnxn_image"
icons(2) = "Pathname_to_toolbar_nnnnnn_image"
mainIcons(0) = "Pathname_to_nxn_image"
mainIcons(1) = "Pathname_to_nnnxn_image"
mainIcons(2) = "Pathname_to_nnnnnn_image"

cmdGroup.IconList = icons
cmdGroup.MainIconList = mainIcons

Dim menuToolbarOption As Integer = swCommandItemType_e.swMenuItem Or swCommandItemType_e.swToolBarItem

cmdIndex0 = cmdGroup.AddCommandItem2("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", mainItemID1, menuToolbarOption)
cmdIndex1 = cmdGroup.AddCommandItem2("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP", "PMPEnable", mainItemID2, menuTo
olbarOption)

cmdGroup.HasToolBar = True
cmdGroup.HasMenu = True
cmdGroup.Activate()

' Get number of command IDs in the CommandGroup
Debug.Print("Number of command IDs in the CommandGroup is " & iCmdMgr.GetCommandIDsCount(0).ToString())

' Get group data from registry
Dim userIDs As Integer()
Dim objIDs As Object = Nothing
iCmdMgr.GetGroupDataFromRegistry(0, objIDs)
userIDs = DirectCast(objIDs, Integer())
Debug.Print("Command IDs found in the registry:")
For Each ID As Integer In userIDs
    Debug.Print(ID.ToString())
Next

Dim bResult As Boolean

Dim flyGroup As FlyoutGroup

flyGroup = iCmdMgr.CreateFlyoutGroup2(flyoutGroupID, "Dynamic Flyout", "Dynamic Flyout", "Flyout Hint", _
    mainIcons, icons, "FlyoutCallback", "FlyoutEnable")

' Add the FlyoutGroup to the context-sensitive menus of faces in parts
bResult = flyGroup.AddContextMenuFlyout(CInt(swDocumentTypes_e.swDocPART), CInt(swSelectType_e.swSelFACES))
Debug.Print("Context-sensitive menu flyout created for faces in parts: " & bResult.ToString())

' Get the total number of FlyoutGroups in CommandManager
Debug.Print("Number of FlyoutGroups is " & iCmdMgr.NumberOfFlyoutGroups)

' Get the FlyoutGroups
Dim objGroups As Object()
objGroups = DirectCast(iCmdMgr.GetFlyoutGroups(), Object())
Debug.Print("Find all FlyoutGroups in CommandManager:")
Dim i As Integer
For i = 0 To objGroups.GetUpperBound(0)
    Debug.Print("FlyoutGroup found")
Next

' Get a FlyoutGroup by its user-defined ID
Dim fogr As IFlyoutGroup
fogr = iCmdMgr.GetFlyoutGroup(91)
Debug.Print(" CmdID: " & fogr.CmdID)
Debug.Print(" Button count: " & fogr.ButtonCount)
Debug.Print(" Flyout type: " & fogr.FlyoutType)
mainIcons = fogr.MainIconList
Debug.Print(" Small button: " & mainIcons(0))
Debug.Print(" Medium button: " & mainIcons(1))
Debug.Print(" Large button: " & mainIcons(2))
icons = fogr.IconList
Debug.Print(" Small toolbar button: " & icons(0))
Debug.Print(" Medium toolbar button: " & icons(1))
Debug.Print(" Large toolbar button: " & icons(2))

For Each docType As Integer In docTypes
    Dim cmdTab As ICommandTab = iCmdMgr.GetCommandTab(docType, Title)

    If Not cmdTab Is Nothing And Not getDataResult Or ignorePrevious Then ' If tab exists, but the registry info is ignored, then re-
create the tab; otherwise the IDs won't match and the tab will be blank
        Dim res As Boolean = iCmdMgr.RemoveCommandTab(cmdTab)
        cmdTab = Nothing
    End If

    If cmdTab Is Nothing Then
        cmdTab = iCmdMgr.AddCommandTab(docType, Title)

        Dim cmdBox As CommandTabBox = cmdTab.AddCommandTabBox

        Dim cmdIDs(3) As Integer
        Dim TextType(3) As Integer

        cmdIDs(0) = cmdGroup.CommandID(cmdIndex0)
        TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        cmdIDs(1) = cmdGroup.CommandID(cmdIndex1)
        TextType(1) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        cmdIDs(2) = cmdGroup.ToolbarID

```

```

        TextType(2) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

        bResult = cmdBox.AddCommands(cmdIDs, TextType)

        Dim cmdBox1 As CommandTabBox = cmdTab.AddCommandTabBox()
        ReDim cmdIDs(1)
        ReDim TextType(1)

        cmdIDs(0) = flyGroup.CmdID
        TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow

        bResult = cmdBox1.AddCommands(cmdIDs, TextType)

        cmdTab.AddSeparator(cmdBox1, cmdIDs(0))

    End If
Next

thisAssembly = Nothing

End Sub

Public Sub RemoveCommandMgr()
    Try
        iBmp.Dispose()
        iCmdMgr.RemoveCommandGroup(mainCmdGroupID)
        iCmdMgr.RemoveFlyoutGroup(flyoutGroupID)
    Catch e As Exception
    End Try
End Sub

Function AddPMP() As Boolean
    ppage = New UserPMPPage
    ppage.Init(iSwApp, Me)
End Function

Function RemovePMP() As Boolean
    ppage = Nothing
End Function

Function CompareIDs(ByVal storedIDs() As Integer, ByVal addinIDs() As Integer) As Boolean

    Dim storeList As New List(Of Integer)(storedIDs)
    Dim addinList As New List(Of Integer)(addinIDs)

    addinList.Sort()
    storeList.Sort()

    If Not addinList.Count = storeList.Count Then

        Return False
    Else

        For i As Integer = 0 To addinList.Count - 1
            If Not addinList(i) = storeList(i) Then

                Return False
            End If
        Next
    End If

    Return True
End Function
#End Region

#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()

    ' Listen for events on all currently open docs
    AttachEventsToAllDocuments()
End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    ' Close events on all currently open docs
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        ' Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            docHandler.DetachEventHandlers() ' This also removes the pair from the hash
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    Try
        AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub DetachSWEvents()
    Try
        RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try

```

```

End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    ' TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function
#End Region

#Region "UI Callbacks"
Sub CreateCube()
    ' Make sure a part open is open
    Dim partTemplate As String
    Dim model As ModelDoc2
    Dim featMan As FeatureManager

    partTemplate = iSwApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
    If Not partTemplate = "" Then
        model = iSwApp.NewDocument(partTemplate, swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0)

        Dim sketchMgr As SketchManager
        sketchMgr = model.SketchManager
        sketchMgr.InsertSketch(True)
        sketchMgr.CreateCornerRectangle(0, 0, 0, 0.1, 0.1, 0.1)

        ' Extrude the sketch
        featMan = model.FeatureManager
        featMan.FeatureExtrusion3(True, _
            False, False, _
            swEndConditions_e.swEndCondBlind, swEndConditions_e.swEndCondBlind, _
            0.1, 0.0, _
            False, False, _
            False, False, _
            0.0, 0.0, _
            False, False, _
            False, False, _
            False, _
            False, True, swStartConditions_e.swStartSketchPlane, 0.0, False)
    Else
        System.Windows.Forms.MessageBox.Show("There is no part template available. Please check your options and make sure there is a part template selected, or select a new part template.")
    End If
End Sub

Sub ShowPMP()
    If Not ppage Is Nothing Then
        ppage.Show()
    End If
End Sub

Function PMPEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        PMPEnable = 0
    Else
        PMPEnable = 1
    End If
End Function

Sub FlyoutCallback()
    Dim flyoutGroup As FlyoutGroup = iCmdMgr.GetFlyoutGroup(flyoutGroupID)
    flyoutGroup.RemoveAllCommandItems()

    flyoutGroup.AddCommandItem("FlyoutCommand 1", "FlyoutCommand 1", 0, "FlyoutCommandItem1", "FlyoutEnableCommandItem1")

```

```

flyGroup.AddCommandItem("FlyoutCommand 2", "FlyoutCommand 2", 0, "FlyoutCommandItem2", "FlyoutEnableCommandItem2")
flyGroup.FlyoutType = CInt(swCommandFlyoutStyle_e.swCommandFlyoutStyle_LastUsed)

End Sub

Public Sub FlyoutCommandItem1()
    Debug.Print("FlyoutCommand 1 called")
End Sub

Public Function FlyoutEnableCommandItem1() As Integer
    Return 1
End Function

Public Sub FlyoutCommandItem2()
    Debug.Print("FlyoutCommand 2 called")
End Sub

Public Function FlyoutEnableCommandItem2() As Integer
    Return 1
End Function

Function FlyoutEnable() As Integer
    If cmdGroup.HasEnabledButton Then
        Return 1
    Else
        Return 0
    End If
End Function

#End Region

End Class

```

Create Force Feature Example (VBA)

This example shows how to create a force feature for use with Basic Motion and Motion Analysis studies.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\motionstudies\weldingrobot\weldingrobot.sldasm.
' 2. Click View > User Interface > MotionManager in SOLIDWORKS Design
'    if the Motion Study 1 tab is not displayed.
' 3. Add a reference to the SOLIDWORKS MotionStudy type library in
'    the Microsoft Visual Basic for Applications IDE (click
'    Tools > SolidWorks version MotionStudy Type Library > OK).
' 4. Open the Immediate window.
'
'
' Postconditions:
' 1. Gets the MotionManager.
' 2. Gets and activates Motion Study 1.
' 3. Selects a face.
' 4. Creates the definition for the Force feature.
' 5. Defines the Force feature.
' 6. Gets the component to which the selected face belongs.
' 7. Selects another face on the component.
' 8. Creates Force1.
' 9. Examine the Immediate window and the MotionManager.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swMotionMgr As SwMotionStudy.MotionStudyManager
    Dim swMotionStudy1 As SwMotionStudy.MotionStudy
    Dim swForceFeat As SldWorks.SimulationForceFeatureData
    Dim boolstatus As Boolean
    Dim swFeat As SldWorks.Feature

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    If (swMotionMgr Is Nothing) Then
        End
    End If

    ' Get Motion Study 1
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")
    If (swMotionStudy1 Is Nothing) Then
        MsgBox "Motion Study 1 is not available."
        End
    End If

    ' Activate Motion Study 1
    swMotionStudy1.Activate

    ' Select a face
    boolstatus = swModelDocExt.SelectByID("", "FACE", 2.07474494005737E-02, -3.42078458101582E-02, -0.137793984025564, False, 0, Nothing, 0)

    ' Create the definition for the Force feature
    Set swForceFeat = swMotionStudy1.CreateDefinition(swFmAEMLinearForce)
    If swForceFeat Is Nothing Then
        Debug.Print "ERROR: Creation of Force feature data object failed."
        Exit Sub
    End If

    ' Define the Force feature
    swForceFeat.ActionType = swSimulationForceAction_ActionOnly
    Set swSelMgr = swModel.SelectionManager

    ' Get the selected face
    swForceFeat.ActionLocation = swSelMgr.GetSelectedObject6(1, -1)

    ' Get the component to which the selected face belongs
    Dim RelObj As Component2
    Set RelObj = swSelMgr.GetSelectedObjectsComponent3(1, -1)

    ' Select a face on a component and create the Force feature relative to this component
    boolstatus = swModelDocExt.SelectByID("", "FACE", 3.29445368450365E-02, -2.94946206198006E-02, -9.99182361467206E-02, False, 0, Nothing, 0)
    swForceFeat.ReferenceComponent = RelObj
    Set swFeat = swMotionStudy1.CreateFeature(swForceFeat)

    If swFeat Is Nothing Then
        Debug.Print " ERROR: Creation of the Force feature failed."
    Else
        Debug.Print "Name of Force feature : " & swFeat.Name
    End If

End Sub

```

Create Forming Tool Example (C#)

This example shows how to create a forming tool for a sheet metal part.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\formingTool.sldprt
//
// Postconditions:
// 1. Creates FormTool1 with the specified stopping face, face to remove,
//    and insertion point.
// 2. Examine the FeatureManager design tree and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateFormTool_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;

        bool boolstatus;
        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            //Select the stopping face: Mark = 1
            boolstatus = Part.Extension.SelectByID("", "FACE", 0.0283459459495816, 0.0187563215566229, 0, true, 1, null, 0);
            //Select the face to remove: Mark = 2
            boolstatus = Part.Extension.SelectByID("", "FACE", 0.0358143533097106, 0.0308169322714775, 0.0066335048657038, true,
2, null, 0);
            //Create a forming tool with insertion point x=0, y=0
            myFeature = Part.FeatureManager.CreateFormTool2(0, 0);
        }

        public SldWorks swApp;
    }
}
```

Create Forming Tool Example (VBA)

This example shows how to create a forming tool for a sheet metal part.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\formingTool.sldprt.  
'-----  
' Postconditions:  
' 1. Creates FormTool1 with the specified stopping face, face to remove,  
'    and insertion point.  
' 2. Examine the FeatureManager design tree and graphics area.  
'-----  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim myFeature As SldWorks.Feature  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    'Select the stopping face: Mark = 1  
    boolstatus = Part.Extension.SelectByID("", "FACE", 2.83459459495816E-02, 1.87563215566229E-02, 0, True, 1, Nothing, 0)  
    'Select the face to remove: Mark = 2  
    boolstatus = Part.Extension.SelectByID("", "FACE", 3.58143533097106E-02, 3.08169322714775E-02, 6.6335048657038E-03, True, 2,  
    Nothing, 0)  
    'Create a forming tool with insertion point x=0, y=0  
    Set myFeature = Part.FeatureManager.CreateFormTool2(0, 0)  
End Sub
```

Create Forming Tool Example (VB.NET)

This example shows how to create a forming tool for a sheet metal part.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\formingTool.sldprt.
'
' Postconditions:
' 1. Creates FormTool1 with the specified stopping face, face to remove,
'    and insertion point.
' 2. Examine the FeatureManager design tree and graphics area.
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim myFeature As Feature
    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.ActiveDoc
        'Select the stopping face: Mark = 1
        boolstatus = Part.Extension.SelectByID("", "FACE", 0.0283459459495816, 0.0187563215566229, 0, True,
1, Nothing, 0)
        'Select the face to remove: Mark = 2
        boolstatus = Part.Extension.SelectByID("", "FACE", 0.0358143533097106, 0.0308169322714775, 0.0066335048657038, True, 2, Nothing, 0)

        'Create a forming tool with insertion point x=0, y=0
        myFeature = Part.FeatureManager.CreateFormTool2(0, 0)

    End Sub

    Public swApp As SldWorks

End Class
```

Create Forming Tool Feature Example (C#)

This example shows how to create a forming tool feature.

// -----

// Preconditions:

// 1. Open *public_documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\sheetmetal\formtools\cover.sldprt*.

// 2. On the SOLIDWORKS Design toolbar, click **Tools > Macro > New** and create a C# macro.

// 3. Copy and paste the code below into **SolidWorksMacro.cs**.

// 4. Modify the two formtoolpath locations.

// 5. Run the macro.

// 6. At the pause, inspect the FeatureManager design tree and the graphics area for the drafted rectangular emboss forming tool.

// Then select a flat face where to re-position (and replace) the forming tool.

// 7. Press F5 finish.

//

// Postconditions:

// 1. A dimple emboss replaces the drafted rectangular emboss.

// 2. Inspect the FeatureManager design tree, graphics area, and the Immediate window.

//

// NOTE: Do not save the part as it may be used elsewhere.

// -----

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace CreateFormingTool_CSharp
```

```
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private FeatureManager featMgr;
        private bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Select a flat face where to insert an emboss forming tool
            boolstatus = swModel.Extension.SelectByRay(0.0553557395141979, 0.0303381093343091, 0.0266948404126879,
            -0.707061513640545, -0.415946801562901, -0.57188484347632, 0.000532844353755021, 2, false, 0, 0);

            featMgr = swModel.FeatureManager;

            LibraryFormToolFeatureData LibFormToolFeatureData;
            LibFormToolFeatureData =
            (LibraryFormToolFeatureData)featMgr.CreateDefinition((int)swFeatureNameID_e.swFmFormToolInstance);

            LibFormToolFeatureData.LinkToFormTool = true;
            LibFormToolFeatureData.FormToolPath = @"E:\images\2025_SP02_release64\d250219.026.F63\WinRel64\WinRel64\data\Design
            Library\forming tools\embosses\drafted rectangular emboss.sldftp";
            LibFormToolFeatureData.OverrideDocumentSettings(true, true, true, true);
            LibFormToolFeatureData.RotationAngle = 0.5;

            Feature swFeat;
```

```

swFeat = featMgr.CreateFeature(LibFormToolFeatureData);

System.Diagnostics.Debugger.Break(); // Inspect the graphics area and FeatureManager design tree; select a flat face
where to re-position (and replace) the forming tool

SelectionMgr selMgr;
selMgr = (SelectionMgr)swModel.SelectionManager;

Face2 face2;
face2 = (Face2)selMgr.GetSelectedObject6(1, -1);

object pickPt;
pickPt = selMgr.GetSelectionPoint2(1, -1);

MathUtility MU;
MU = (MathUtility)swApp.GetMathUtility();

MathPoint mathPt;
mathPt = (MathPoint)MU.CreatePoint(pickPt);

swModel.ClearSelection2(true);

LibraryFormToolFeatureData swLibFormFeatData1;
swLibFormFeatData1 = (LibraryFormToolFeatureData)swFeat.GetDefinition();

Debug.Print("Forming tool punch ID: " + swLibFormFeatData1.GetPunchID());
Debug.Print("Forming tool part configuration: " + swLibFormFeatData1.FormToolConfiguration);
Debug.Print("Forming tool path: " + swLibFormFeatData1.FormToolPath);
Debug.Print("Link to Design Library part? " + swLibFormFeatData1.LinkToFormTool);
Debug.Print("Rotation angle: " + swLibFormFeatData1.RotationAngle);

swLibFormFeatData1.AccessSelections(swModel, null);

Face2 face;
face = (Face2)swLibFormFeatData1.PlacementFace;

swLibFormFeatData1.RotationAngle = 0.7833;
swLibFormFeatData1.OverrideDocumentSettings(true, true, true, true);
swLibFormFeatData1.FormToolConfiguration = "Default";
swLibFormFeatData1.FormToolPath = @"E:\images\2025_SP02_release64\d250219.026.F63\WinRel64\WinRel64\data\Design
Library\forming tools\embosses\dimple.SLDFTP";
swLibFormFeatData1.LinkToFormTool = true;
Entity ent;
ent = (Entity)face;
ent.Select4(true, null);

swLibFormFeatData1.SetPlacementFace(face2, mathPt);

swFeat.ModifyDefinition(swLibFormFeatData1, swModel, null);

Face2 face1;
face1 = (Face2)swLibFormFeatData1.PlacementFace;
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```